

TARZAN: A Region-Based Library for Forward and Backward Reachability of Timed Automata (Extended Version)*

Andrea Manini^[0009–0009–6103–9244], Matteo Rossi^[0000–0002–9193–9560], and
Pierluigi San Pietro^[0000–0002–2437–8716]

Politecnico di Milano
firstname.lastname@polimi.it

Abstract. The zone abstraction, widely adopted for its notable practical efficiency, is the de facto standard in the verification of Timed Automata (TA). Nonetheless, region-based abstractions have been shown to outperform zones in specific subclasses of TA. To complement and support mature zone-based tools, we introduce TARZAN, a C++ region-based verification library for TA. The algorithms implemented in TARZAN use a novel region abstraction that tracks the order in which clocks become unbounded. This additional ordering induces a finer partitioning of the state space, enabling backward algorithms to avoid the combinatorial explosion associated with enumerating all ordered partitions of unbounded clocks, when computing immediate delay predecessor regions. We validate TARZAN by comparing forward reachability results against the state-of-the-art tools Uppaal and TChecker. The experiments confirm that zones excel when TA have large constants and strict guards. In contrast, TARZAN exhibits superior performance on closed TA and TA with punctual guards. Finally, we demonstrate the efficacy of our backward algorithms, establishing a foundation for region-based analysis in domains like Timed Games, where backward exploration is essential.

1 Introduction

Formally verifying the behavioral correctness of real-time systems (also including distributed systems) is essential for ensuring that critical properties hold. Since their introduction, Timed Automata (TA) have become one of the most widely adopted formalisms for modeling and analyzing such systems [15,31].

A key challenge in the verification of TA is their infinite state space, addressed by the *region* [1] abstraction, which provides a finite representation enabling algorithmic verification. Alternatives include *Binary Decision Diagrams* [35] and *zones* [18], the latter requiring *normalization* for state space finiteness [9]. Zones, typically implemented using *Difference Bound Matrices (DBMs)* [16], are widely used in tools such as Uppaal [6], TChecker [19], Kronos [13], and Synthia [28].

* The experimental results presented in this paper are reproducible using the artifact available on Zenodo [27].

Zones have shown to be highly efficient in practice and, over the last decades, have become the de facto standard for the formal verification of TA, although they have well-known limitations [11,24,36]. Yet, prior work has demonstrated that for the class of *closed* TA, *i.e.*, TA containing only non-strict guards, region-based abstractions may outperform state-of-the-art zone-based tools [21].

The promising performance of region-based abstractions on appropriate subclasses of TA motivated the development of TARZAN (Timed Automata Region and Zone library for real-time systems ANalysis) [26], a C++20 library designed to support and complement state-of-the-art zone-based tools for the formal verification of TA. Currently, TARZAN supports only region-based reachability of TA; optimizations leveraging zones are deferred to future work. The implementation of backward reachability algorithms enables the extension of TARZAN to domains in which backward analysis is essential, such as Timed Games [25].

When compared against zone-based state-of-the-art tools including Uppaal and TChecker, TARZAN exhibits superior performance on TA and networks of TA featuring *punctual* guards, *i.e.*, guards with equality constraints, and on closed TA, even with large constants or many clocks. However, the size of constants remains a significant bottleneck in other classes of TA (*e.g.*, zones excel in TA with strict guards). Moreover, since TARZAN only supports reachability analysis, using forward reachability to verify safety properties for unreachable regions requires exploring the entire reachable state space. We show in this paper that backward reachability can enhance the verification of safety properties.

Related works Abstractions closely related to regions have already been proposed to better capture the semantics of TA. In [37], regions are encoded as Boolean formulae via discrete interpretations. The *slot-based* abstraction proposed in [2] groups regions by time using integer-bounded intervals over a global clock. A data structure, called *time-dart*, is introduced in [21] to compactly encode configurations of TA and their time successors without explicit enumeration. In [10], regions of bounded TA are represented as tuples combining integer clock values and a partition of clocks that reflects the ordering of their fractional parts. The region abstraction is extended in [12] to *durational-action* TA.

To the best of our knowledge, no region-based library for TA verification, nor any libraries or tools based on the aforementioned abstractions, currently exist. Consequently, we compare TARZAN against the state-of-the-art tools Uppaal and TChecker. Existing libraries that explicitly support the analysis of TA using DBM-based zone representations include [30,33]. Other libraries offer more general polyhedral abstractions beyond zones, such as those provided by [3,29].

Contributions The main contribution of this work is two-fold: (i) we first introduce a refined region representation that tracks the order in which clocks become unbounded. While this representation may increase the total number of regions by inducing a finer partitioning of the state space, a major advantage is that it guarantees at most three immediate delay predecessors for any region; (ii) building on this new representation, we develop and implement algorithms in TARZAN for computing delay and discrete successors and predecessors of regions. Existing tools may integrate TARZAN to enhance their verification capabilities.

This paper is organized as follows: Section 2 introduces the necessary theoretical background. Section 3 presents the proposed new region representation, from which a classification of regions is derived. Section 4 details the algorithms (and their pseudocode) to compute discrete successors and immediate delay predecessors of regions using this representation. Section 5 presents an overview of TARZAN and provides an empirical evaluation of forward and backward reachability. Section 6 concludes and outlines future developments. An appendix provides additional algorithms, proofs, and full experimental results.

2 Theoretical Background

The timing behavior of real-time systems is naturally modeled by the formalism of TA, which is obtained by extending classical finite-state automata with real-valued clocks. In the following, we use the term action to refer to the events that a real-time system can exhibit. TA are defined as follows [1]:

Definition 1. *A Timed Automaton $\mathcal{A}$ is a tuple $\mathcal{A} = (Act, Q, Q_0, X_{\mathcal{A}}, T, Inv)$, where Act is a finite set of actions, Q is a finite set of locations, $Q_0 \subseteq Q$ is a set of initial locations, $X_{\mathcal{A}}$ is a finite set of clocks, $T \subseteq Q \times Act \times \Gamma(X_{\mathcal{A}}) \times 2^{X_{\mathcal{A}}} \times Q$ is a transition relation, and $Inv : Q \rightarrow \Gamma(X_{\mathcal{A}})$ is an invariant-assignment function.*

Let $\mathcal{A}$ be a TA. Clocks are special variables that can only be reset or compared against non-negative integer constants. The value of all clocks is initialized to zero and grows with derivative 1 until a reset occurs. When a clock is reset, its value becomes zero; after a reset, it starts increasing again. $\Gamma(X_{\mathcal{A}})$ is the set of clock guards, i.e., conditions over clocks that must be satisfied for a transition to fire. Clock guards are defined by the following grammar: $\gamma := \xi \mid \gamma \wedge \gamma$, where $\xi := x \sim c$ is a clock constraint, $x \in X_{\mathcal{A}}$, $c \in \mathbb{N}$, and $\sim \in \{\leq, <, =, >, \geq\}$. We write $\xi \in \gamma$ to indicate that the clock constraint ξ occurs within the guard γ . For instance, if $\gamma := x_1 < c \wedge x_2 \geq c$, then $\xi_1 := x_1 < c$ and $\xi_2 := x_2 \geq c$, where $\xi_1, \xi_2 \in \gamma$. The powerset $2^{X_{\mathcal{A}}}$ indicates that a transition may reset a subset of the clocks. We denote by $c_m \in \mathbb{N}$ the maximum constant appearing in $\mathcal{A}$ and assume, without loss of generality, that it is the same for all clocks of $\mathcal{A}$.

A pair (q, v) is a configuration of a TA $\mathcal{A}$, where $q \in Q$ and $v : X_{\mathcal{A}} \rightarrow \mathbb{R}_{\geq 0}$ is a clock valuation function. We denote by $Eval_{\mathcal{A}}$ the set of all clock valuations of $\mathcal{A}$. Given a subset $Y \subseteq X_{\mathcal{A}}$ of clocks, we denote by $v[Y \leftarrow 0]$ the clock valuation defined as $v[Y \leftarrow 0](x) = 0$ if $x \in Y$ and $v[Y \leftarrow 0](x) = v(x)$ otherwise. Given a constant $\delta \in \mathbb{R}_{\geq 0}$ and a clock $x \in X_{\mathcal{A}}$, the value $(v + \delta)(x)$ is defined as $v(x) + \delta$.

The transition system $TS(\mathcal{A}) = (S, Act \cup \mathbb{R}_{\geq 0}, I, \hookrightarrow)$ captures the semantics of a TA $\mathcal{A}$, where $S = Q \times Eval_{\mathcal{A}}$, $I = \{(q, v) \mid q \in Q_0 \wedge \forall x \in X_{\mathcal{A}} : v(x) = 0\}$, and $\hookrightarrow$ is the transition relation containing: (i) discrete transitions $((q, v), a, (q', v'))$ if there exists a transition $(q, a, \gamma, Y, q') \in T$ in $\mathcal{A}$ such that v satisfies both γ and $Inv(q)$, and $v'(x) = v[Y \leftarrow 0](x)$ satisfies $Inv(q')$, where $a \in Act$, for all $x \in X_{\mathcal{A}}$, and (ii) delay transitions $((q, v), \delta, (q, v'))$ if there exists a constant $\delta \in \mathbb{R}_{\geq 0}$ such that $v'(x) = (v + \delta)(x)$ satisfies $Inv(q)$, for all $x \in X_{\mathcal{A}}$. A full description of the standard transition system semantics of TA is covered in [4, Chapter 9].

The infinite state space of TA can be conveniently analyzed by resorting to a finite abstraction, known as the *region* abstraction [1]. For any real number $m \in \mathbb{R}$, we denote its integral and fractional parts by $\lfloor m \rfloor$ and $fr(m)$, respectively.

Definition 2 (Clock equivalence relation). Let $\mathcal{A} = (Act, Q, Q_0, X_{\mathcal{A}}, T, Inv)$ be a TA. Two clock valuations $v, v' \in Eval_{\mathcal{A}}$ are called clock-equivalent, denoted by $v \cong v'$, if, and only if, it either holds that: (i) $v(x) > c_m$ and $v'(x) > c_m$, for all $x \in X_{\mathcal{A}}$, or (ii) for any $x, y \in X_{\mathcal{A}}$, with $v(x), v'(x) \leq c_m$ and $v(y), v'(y) \leq c_m$, it holds that: (ii_a) $\lfloor v(x) \rfloor = \lfloor v'(x) \rfloor$ and $fr(v(x)) = 0 \Leftrightarrow fr(v'(x)) = 0$, and (ii_b) $fr(v(x)) \sim fr(v(y)) \Leftrightarrow fr(v'(x)) \sim fr(v'(y))$, where $\sim \in \{<, =\}$.

Definition 3 (Region). Let $\mathcal{A}$ be a TA and $s = (q, v)$ be a configuration of $\mathcal{A}$. The clock region $[v]$ of $v \in Eval_{\mathcal{A}}$ is the set of all clock valuations equivalent to v under the relation of Definition 2. The state region $[s]$ of s is: $[s] = (q, [v]) = \{(q, v') \mid v' \in [v]\}$; $[s]$ is initial iff $q \in Q_0$ and $\forall v' \in [v], \forall x \in X_{\mathcal{A}} : v'(x) = 0$.

In the remainder of this work, we focus on state regions, referred to simply as *regions*. Let $\mathcal{A}$ be a TA and $[s] = (q, [v])$ a region of $\mathcal{A}$. A region $[s'] = (q', [v'])$ is an *immediate delay successor* of $[s]$ if either: (i) $q = q' \wedge [v] = [v']$ and, for all $\bar{v} \in [v]$ and $x \in X_{\mathcal{A}}$, $\bar{v}(x) > c_m$, or (ii) $q = q' \wedge [v] \neq [v']$ and for all $\bar{v} \in [v]$ there exists $\delta \in \mathbb{R}_{>0}$ such that $\bar{v} + \delta \in [v'] \wedge \forall 0 \leq \delta' \leq \delta : \bar{v} + \delta' \in ([v] \cup [v'])$. A region $[s']$ is a *delay successor* of $[s]$ if there exists a finite sequence of regions $[s_0], [s_1], \dots, [s_n]$ such that $[s_0] = [s]$, $[s_n] = [s']$, and each $[s_{i+1}]$ is an immediate delay successor of $[s_i]$, for $0 \leq i < n$. A region $[s']$ is a *discrete successor* of $[s]$ if $[v'] = [v[Y \leftarrow 0]]$, where $Y \subseteq X_{\mathcal{A}}$ is the subset of clocks reset over the discrete transition from $[s]$ to $[s']$. Symmetric definitions can be given for *immediate delay predecessors*, *delay predecessors*, and *discrete predecessors*.

3 Representing Regions with Ordered Clock Partitions

We introduce a new region representation that captures the order in which clocks become unbounded (exceeding c_m). For bounded clocks (with value up to c_m), only the ordering of their fractional part is preserved. Our representation builds on [10], which focuses on TA with bounded clocks. By also handling unbounded clocks, we significantly extend the applicability of region-based state space exploration to a broader class of TA. While forward exploration remains unaffected, our representation influences backward exploration (Theorem 1 and Theorem 2).

Let $\mathcal{A} = (Act, Q, Q_0, X_{\mathcal{A}}, T, Inv)$ be a TA, $q \in Q$. Furthermore, let $\ell, \tau \in \mathbb{N}$ be two integers such that $\ell + \tau \leq |X_{\mathcal{A}}|$. We denote by $X_i \subseteq X_{\mathcal{A}}$ a subset of clocks of $\mathcal{A}$, for $-\ell \leq i \leq \tau$. We require that these subsets induce a partition of $X_{\mathcal{A}}$, i.e., $X_{\mathcal{A}} = \bigcup_i X_i$ and, for all indices $-\ell \leq i, j \leq \tau$ such that $i \neq j$, $X_i \cap X_j = \emptyset$. Finally, we introduce a function $h : X_{\mathcal{A}} \rightarrow \{0, \dots, c_m\}$ that assigns an integer value to each clock in $X_{\mathcal{A}}$. A region of $\mathcal{A}$ can be represented as follows:

$$\mathcal{R} := \{q, h, X_{-\ell}, X_{-(\ell-1)}, \dots, X_{-1}, X_0, X_1, \dots, X_{(\tau-1)}, X_{\tau}\}. \quad (1)$$

Set X_0 is always present even if empty; the other sets are omitted when empty. The sets $X_{-\ell}, \dots, X_{-1}$ represent unbounded clocks, capturing the ordering in which these clocks become unbounded (*e.g.*, all clocks in X_{-1} become unbounded simultaneously and prior to all clocks in X_{-2}). Conversely, the sets $X_1, \dots, X_{\mathfrak{r}}$ represent bounded clocks, capturing the ordering of their fractional parts (*e.g.*, the fractional parts of all clocks in X_1 are equal and less than the fractional parts of all clocks in X_2). X_0 contains bounded clocks that have no fractional part (we also say these clocks *are in the unit*). The integers ℓ and $\mathfrak{r}$ denote the number of sets containing unbounded and bounded clocks, respectively ($\mathfrak{r}$ does not consider X_0). Their values are automatically updated whenever a set is inserted into or removed from $\mathcal{R}$. We use a subscript to disambiguate between different regions: for instance, $q_{\mathcal{R}}$ refers to the location q of $\mathcal{R}$, $X_{\mathcal{R},0}$ to the set X_0 of $\mathcal{R}$, and $\ell_{\mathcal{R}}$ indicates the number of sets containing unbounded clocks in $\mathcal{R}$.

For conciseness, we also refer to the elements of a region $\mathcal{R}$ as its *structure*. Two regions $\mathcal{R}, \mathcal{R}'$ are *structurally equivalent* if the following holds: $q_{\mathcal{R}} = q_{\mathcal{R}'} \wedge \ell_{\mathcal{R}} = \ell_{\mathcal{R}'} \wedge \mathfrak{r}_{\mathcal{R}} = \mathfrak{r}_{\mathcal{R}'}$ and, for all $-\ell_{\mathcal{R}} \leq i \leq \mathfrak{r}_{\mathcal{R}}$, $X_{\mathcal{R},i} = X_{\mathcal{R}',i}$.

Example 1. Let $\mathcal{A}$ be a TA with clocks $X_{\mathcal{A}} = \{x, y, z, w\}$ and $c_m = 5$. Consider the region defined as: $\{(x, y, z, w) \mid 2 < x < 3, 2 < y < 3, z > 5, w > 5, x = y\}$, *i.e.*, a region (assumed to be in $q \in Q$) in which x and y are bounded and equal (they have the same fractional part), while clocks z and w are unbounded (we assume that z became unbounded before w). The region is represented using Equation (1) as: $\mathcal{R} = \{q, h(x) = 2, h(y) = 2, h(z) = 5, h(w) = 5, X_{-2}, X_{-1}, X_0, X_1\}$, where $X_{-2} = \{w\}, X_{-1} = \{z\}, X_0 = \emptyset, X_1 = \{x, y\}, \ell = 2$, and $\mathfrak{r} = 1$.

Definition 4 (Region class). *If the order in which clocks become unbounded is tracked, regions can be classified in four distinct classes—Z (Zero), P (Positive), M (Mixed), and U (Unbounded)—defined as:* (i) *Class Z:* $\ell \geq 0 \wedge \mathfrak{r} = 0 \wedge X_0 \neq \emptyset$; (ii) *Class P:* $\ell \geq 0 \wedge \mathfrak{r} > 0 \wedge X_0 = \emptyset$; (iii) *Class M:* $\ell \geq 0 \wedge \mathfrak{r} > 0 \wedge X_0 \neq \emptyset$; and (iv) *Class U:* $\ell > 0 \wedge \mathfrak{r} = 0 \wedge X_0 = \emptyset$.

Class *Z* denotes regions in which some clocks may be unbounded, while all bounded clocks have no fractional part and at least one clock must be bounded. Class *P* denotes regions where some clocks may be unbounded, and all bounded clocks have a fractional part greater than zero (again, at least one clock must be bounded). Class *M* denotes regions where some clocks may be unbounded and the bounded clocks are partitioned into those with no fractional part and those with a fractional part greater than zero (at least two clocks must be bounded, one with no fractional part and one with a fractional part greater than zero). Finally, class *U* denotes regions in which all clocks are unbounded. Notice that the region presented in Example 1 belongs to class *P*. This classification into different classes lies at the core of the algorithms that compute the immediate delay successor and immediate delay predecessors of a given region.

Transitioning between classes Let $\mathcal{R}_Z, \mathcal{R}_P, \mathcal{R}_M$, and $\mathcal{R}_U$ be regions of classes *Z*, *P*, *M*, and *U*, respectively. Transitions from a region to its immediate delay successor induce the following deterministic transitions between classes:

- *From $\mathcal{R}_Z$ to $\mathcal{R}_P$* : at least one clock $x \in X_{\mathcal{R}_Z,0}$ must increase such that its fractional part becomes greater than zero, while remaining bounded. Other clocks in $X_{\mathcal{R}_Z,0}$ can either become unbounded or remain bounded and grow at the same rate of x , *i.e.*, all bounded clocks have the same fractional part.
- *From $\mathcal{R}_Z$ to $\mathcal{R}_U$* : all clocks in $X_{\mathcal{R}_Z,0}$ become unbounded simultaneously.
- *From $\mathcal{R}_P$ to $\mathcal{R}_Z$* : it must hold that $\mathfrak{r}_{\mathcal{R}_P} = 1$. All clocks in $X_{\mathcal{R}_P,1}$ must reach the next unit (they will be inserted into $X_{\mathcal{R}_Z,0}$) and their integer value is incremented by 1, as they have a fractional part greater than zero.
- *From $\mathcal{R}_P$ to $\mathcal{R}_M$* : it must hold that $\mathfrak{r}_{\mathcal{R}_P} > 1$. All clocks in $X_{\mathcal{R}_P,\mathfrak{r}_{\mathcal{R}_P}}$ must reach the next unit (they will be inserted into $X_{\mathcal{R}_M,0}$) and their integer value is incremented by 1, as they have the biggest fractional part.
- *From $\mathcal{R}_M$ to $\mathcal{R}_P$* : clocks in $X_{\mathcal{R}_M,0}$ can become unbounded or their fractional part can become greater than zero while remaining bounded.

The missing transitions between classes are explained as follows. By Definition 4, transitioning to $\mathcal{R}_U$ is only possible from $\mathcal{R}_Z$, since $\mathfrak{r}_{\mathcal{R}_U} = 0 \wedge X_{\mathcal{R}_U,0} = \emptyset$ must hold. It is not possible to transition from $\mathcal{R}_Z$ to $\mathcal{R}_M$, since all clocks will exit from $X_{\mathcal{R}_Z,0}$. An immediate delay successor of $\mathcal{R}_M$ has at least one clock with a nonzero fractional part, making it impossible to transition from $\mathcal{R}_M$ to $\mathcal{R}_Z$.

Transitions between classes contain no self-loops; the immediate delay successor of any region is always a distinct region. $\mathcal{R}_U$ does not have delay successors, as clocks can increase indefinitely once unbounded. The structure of regions influences transitioning between classes; *e.g.*, from $\mathcal{R}_M$ with $\mathfrak{r}_{\mathcal{R}_M} = 3$, several cycles between $\mathcal{R}_P$ and $\mathcal{R}_M$ are required before reaching $\mathcal{R}_Z$ via delay transitions; this becomes possible once $\mathfrak{r}_{\mathcal{R}_M}$ decreases to 1 as some clocks become unbounded.

4 Discrete Successors and Immediate Delay Predecessors

We present algorithms computing discrete successors and immediate delay predecessors regions. For brevity, immediate delay successors and discrete predecessors algorithms are deferred to the Appendix in Section A and Section B.

Discrete successors Algorithm 1, whose time complexity is $O(|T| \cdot |X_{\mathcal{A}}|)$, computes the discrete successors of a given region $\mathcal{R}$. In particular, a set storing the resulting discrete successors is initialized in Line 2 and returned in Line 13. The loop in Line 3 iterates over all transitions t of the original TA that exit from the current location $q_{\mathcal{R}}$ of $\mathcal{R}$, identifying all possible discrete successors of $\mathcal{R}$. If the guard γ of t is satisfied by combining the integer values of the clocks of $\mathcal{R}$, as returned by $h_{\mathcal{R}}$, with their fractional parts (Line 4), a copy $\mathbf{R}$ of $\mathcal{R}$ is created to represent the successor region (Line 5). The location of $\mathbf{R}$ is updated to the target location q of t (Line 6). The loop in Line 7 iterates over all integers $-\ell_{\mathbf{R}} \leq i \leq \mathfrak{r}_{\mathbf{R}}$, enabling the inner loop in Line 8 to examine each clock set of $\mathbf{R}$. If a clock x is reset in t , its integer value is set to zero (Line 9). Moreover, if x is either unbounded or has a nonzero fractional part (Line 10), it is inserted into $X_{\mathbf{R},0}$ and removed from its previous clock set (Line 11). Finally, the resulting region $\mathbf{R}$ is added to the result set in Line 12.

Algorithm 1: find-discrete-successors($\mathcal{R}$)

Data: $\mathcal{R}$: a region.
Result: The discrete successors of $\mathcal{R}$.

```

1 begin
2    $res \leftarrow \emptyset$ ;
3   foreach  $t = (q_{\mathcal{R}}, a, \gamma, Y, q) \in T$  do
4     if  $\gamma$  is satisfied by  $\mathcal{R}$  then
5        $\mathbf{R} \leftarrow$  copy of  $\mathcal{R}$ ;
6        $q_{\mathbf{R}} \leftarrow q$ ;
7       for  $i = -\ell_{\mathbf{R}}$  to  $\tau_{\mathbf{R}}$  do
8         foreach  $x \in X_{\mathbf{R},i}$  such that  $x \in Y$  do
9            $h_{\mathbf{R}}(x) \leftarrow 0$ ;
10          if  $i \neq 0$  then
11             $X_{\mathbf{R},0} \leftarrow X_{\mathbf{R},0} \cup \{x\}$ ;  $X_{\mathbf{R},i} \leftarrow X_{\mathbf{R},i} \setminus \{x\}$ ;
12         $res \leftarrow res \cup \{\mathbf{R}\}$ ;
13 return  $res$ ;

```

Immediate delay predecessors The following theorem (whose proof is shown in the Appendix, Section C) establishes that, when tracking the order in which clocks become unbounded, the number of immediate delay predecessors of a given region does not exhibit exponential blow-up.

Theorem 1. *If the order in which clocks become unbounded is tracked, the number of immediate delay predecessors of a given region is at most three.*

Indeed, if the order in which clocks become unbounded is not tracked, computing the immediate delay predecessors of regions with unbounded clocks would require enumerating all ordered partitions of the unbounded clocks, as every order in which the clocks become unbounded would then be admissible (regions without unbounded clocks would have a unique immediate delay predecessor).

The immediate delay predecessors of a given region $\mathcal{R}$ are computed by Algorithm 2, whose time complexity is $O(|X_{\mathcal{A}}|)$ (we refer to *predecessors* in its description). In particular, Line 2 initializes a set, returned in Line 20, storing the computed predecessors. The algorithm proceeds by analyzing the class of $\mathcal{R}$ to compute its predecessors by modifying a copy $\mathbf{R}$ of $\mathcal{R}$, created in Line 3.

If $\mathcal{R}$ is of class U (Line 4), the predecessor is obtained by inserting set $X_{\mathbf{R},-\ell_{\mathbf{R}}}$, which contains the clocks that became unbounded while exiting from the unit, into $X_{\mathbf{R},0}$. Then, $X_{\mathbf{R},-\ell_{\mathbf{R}}}$ is removed from $\mathbf{R}$ (Line 5).

For regions of class Z or class M (Line 6), an auxiliary set X_{tmp} is initialized (Line 7). The integer part of each clock $x \in X_{\mathbf{R},0}$ (Line 8) is decremented by 1 (Line 9), and x is moved from $X_{\mathbf{R},0}$ to X_{tmp} (Line 10). Since these clocks must have the highest fractional part, X_{tmp} is inserted to the right of $X_{\mathbf{R},\tau_{\mathbf{R}}}$ (Line 11).

If $\mathcal{R}$ is of class P (Line 12), the algorithm considers both the cases where the predecessor is of class Z or class M . First, clocks in $X_{\mathbf{R},1}$ (those that left the

Algorithm 2: find-immediate-delay-predecessors($\mathcal{R}$)

Data: $\mathcal{R}$: a region where no clock x is such that: $h_{\mathcal{R}}(x) = 0 \wedge x \in X_{\mathcal{R},0}$.

Result: The immediate delay predecessors of $\mathcal{R}$.

```

1 begin
2    $res \leftarrow \emptyset$ ;
3    $\mathbf{R} \leftarrow$  copy of  $\mathcal{R}$ ;
4   if  $X_{\mathcal{R},0} = \emptyset \wedge \mathbf{r}_{\mathcal{R}} = 0$  then
5     | insert clocks of  $X_{\mathbf{R},-\ell_{\mathbf{R}}}$  in  $X_{\mathbf{R},0}$  and remove  $X_{\mathbf{R},-\ell_{\mathbf{R}}}$  from  $\mathbf{R}$ ;
6   else if  $X_{\mathcal{R},0} \neq \emptyset$  then
7     |  $X_{tmp} \leftarrow \emptyset$ ;
8     | foreach  $x \in X_{\mathcal{R},0}$  do
9       |    $h_{\mathbf{R}}(x) \leftarrow h_{\mathcal{R}}(x) - 1$ ;
10      |    $X_{tmp} \leftarrow X_{tmp} \cup \{x\}$ ;  $X_{\mathbf{R},0} \leftarrow X_{\mathbf{R},0} \setminus \{x\}$ ;
11      |   insert  $X_{tmp}$  to the right of  $X_{\mathbf{R},\mathbf{r}_{\mathbf{R}}}$ ;
12   else
13     | insert clocks of  $X_{\mathbf{R},1}$  in  $X_{\mathbf{R},0}$  and remove  $X_{\mathbf{R},1}$  from  $\mathbf{R}$ ;
14     | if  $\ell_{\mathcal{R}} \geq 1$  then
15       |    $\mathbf{R}_1 \leftarrow$  copy of  $\mathcal{R}$ ;
16       |   insert clocks of  $X_{\mathbf{R}_1,-\ell_{\mathbf{R}_1}}$  in  $X_{\mathbf{R}_1,0}$  and remove  $X_{\mathbf{R}_1,-\ell_{\mathbf{R}_1}}$  from  $\mathbf{R}_1$ ;
17       |    $\mathbf{R}_2 \leftarrow$  copy of  $\mathbf{R}_1$ ;
18       |   insert clocks of  $X_{\mathbf{R}_2,1}$  into  $X_{\mathbf{R}_2,0}$  and remove  $X_{\mathbf{R}_2,1}$  from  $\mathbf{R}_2$ ;
19       |    $res \leftarrow res \cup \{\mathbf{R}_1, \mathbf{R}_2\}$ ;
20   return ( $res \cup \{\mathbf{R}\}$ );

```

unit) are moved into $X_{\mathbf{R},0}$ (Line 13). Then, Line 14 checks whether unbounded clocks must also be handled. If so, two additional regions are created. The first, $\mathbf{R}_1$, is created on Line 15 as a copy of $\mathcal{R}$ and updated on Line 16 to represent the case in which all clocks with zero fractional part in the predecessor became unbounded. After $\mathbf{R}_1$ is updated, it is copied in $\mathbf{R}_2$ (Line 17), which is modified on Line 18 to capture the case in which only a subset of those clocks became unbounded. Both $\mathbf{R}_1$ and $\mathbf{R}_2$ are collected on Line 19 in the result set.

Example 2. Let $\mathcal{A}$ be a TA having clocks $X_{\mathcal{A}} = \{x, y\}$ and maximum constant $c_m = 2$. Consider the region $\mathcal{R}_0$ as depicted in Figure 1a. Here, triangles are used to symbolically represent entire regions rather than individual points (*e.g.*, the triangle corresponding to $\mathcal{R}_2$ represents the region $\{(x, y) \mid 1 < x < 2 \wedge y > 2\}$). For this reason, they are not drawn on the diagonals, as would be implied by Definition 2. For visual clarity, the figure does not represent the portion of the state space where $y < 1$. Since $\mathcal{R}_0$ lies above the diagonal originating from $(2, 2)$, it corresponds to the case in which y became unbounded before x . $\mathcal{R}_0$ can be represented as: $\mathcal{R}_0 = \{q, h(x) = 2, h(y) = 2, X_{-2} = \{x\}, X_{-1} = \{y\}, X_0 = \emptyset\}$.

We compute the delay predecessors of $\mathcal{R}_0$ using Algorithm 2. For conciseness, we stop the computation when reaching a bounded region or because no further delay predecessors exist. The algorithm yields a single immediate delay predeces-

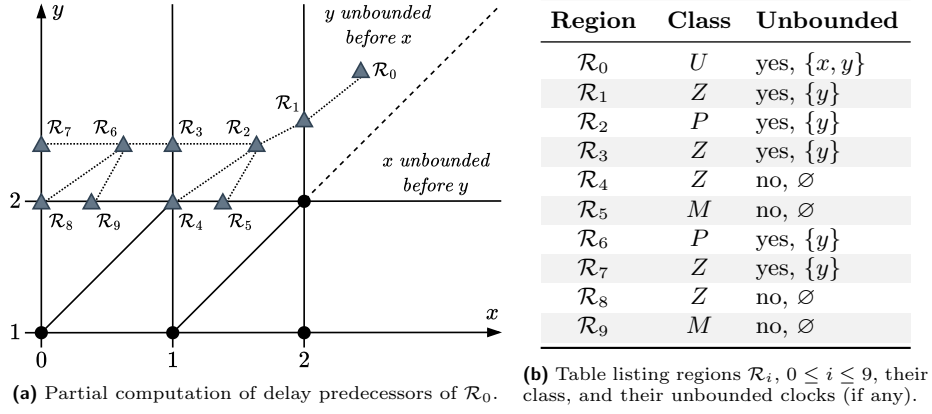


Fig. 1: Example of delay predecessors computation from $\mathcal{R}_0$ (left), stopped upon reaching bounded regions, and a table with additional information about the regions (right). Triangles symbolically represent entire regions rather than specific points within them.

sor of class Z (Figure 1b): $\mathcal{R}_1 = \{q, h(x) = 2, h(y) = 2, X_{-1} = \{y\}, X_0 = \{x\}\}$. Applying the same algorithm to $\mathcal{R}_1$ yields a single immediate delay predecessor: $\mathcal{R}_2 = \{q, h(x) = 1, h(y) = 2, X_{-1} = \{y\}, X_0 = \emptyset, X_1 = \{x\}\}$. Then, from $\mathcal{R}_2$, three immediate delay predecessors can be computed. Two of them ($\mathcal{R}_4$ and $\mathcal{R}_5$) are not further analyzed in this example, as they are already bounded. Consider the region: $\mathcal{R}_3 = \{q, h(x) = 1, h(y) = 2, X_{-1} = \{y\}, X_0 = \{x\}\}$. This region is structurally equivalent to $\mathcal{R}_1$, except that the integer value of x has been decremented by 1. The computation from $\mathcal{R}_1$ applies similarly from $\mathcal{R}_3$.

The computation of delay predecessors can be optimized. Assume that, for a region $\mathcal{R}$, Algorithm 2 can be applied n times. Furthermore, assume that all clocks in $\mathcal{R}$ are bounded and that $\tau_{\mathcal{R}} > 0$ holds. We define the *period* of $\mathcal{R}$ as $\theta = 2 \cdot (\tau_{\mathcal{R}} + 1)$ if $X_{\mathcal{R},0} \neq \emptyset$; $\theta = 2 \cdot \tau_{\mathcal{R}}$ otherwise. Then, decrement the integer value of every clock of $\mathcal{R}$ by $\lfloor n/\theta \rfloor$ (ensuring their integer value does not go below zero), obtaining a new region $\mathcal{R}'$. After this step, if none of the clocks of $\mathcal{R}'$ are equal to zero (with no fractional part), it suffices to apply Algorithm 2 only $n \bmod \theta$ times to $\mathcal{R}'$ to obtain the final result, possibly stopping earlier if one or more clocks become equal to zero during this process. Similar optimizations can also be applied when some clocks are unbounded.

Discrete predecessors (intuition) When computing a discrete successor, a subset of clocks may be reset, and therefore appear with value zero in the successor region. Discrete predecessors computation aims to reverse this operation. However, starting from a region in which the value of some clocks is zero (with no fractional part), a challenge arises, since the original values of these clocks prior to being reset are unknown. As a result, all possible combinations of values for the reset clocks must be considered (including cases where they may have been unbounded) to ensure all discrete predecessors are computed. This process can

be simplified using the guard of the transition over which discrete predecessors are computed, as it may constrain the possible values of clocks prior to reset.

The following theorem (whose proof is shown in the Appendix, Section D) establishes that, when tracking the order in which clocks become unbounded, the number of discrete predecessors of a given region suffers from an inherent exponential blow-up that cannot be avoided in general.

Theorem 2. *Let $\mathcal{A}$ be a TA such that $n = |X_{\mathcal{A}}|$ and let $\mathcal{R}$ be a region of $\mathcal{A}$. If the order in which clocks become unbounded must be tracked, then the number of discrete predecessors of $\mathcal{R}$ over a single transition is $O\left(\left(\frac{c_m+1}{\ln 2}\right)^n \cdot (n+1)!\right)$.*

5 Implementation and Experimental Evaluation

This section introduces TARZAN along with a detailed empirical evaluation of its forward and backward reachability algorithms. To express reachability queries, we use the standard Computation Tree Logic formula $\exists\Diamond(\varphi)$, meaning that there exists a path in the state space in which φ eventually holds true.

5.1 The TARZAN library

TARZAN is a C++20 library for performing forward and backward reachability analysis of TA and networks of TA, using either breadth-first (BFS) or depth-first (DFS) state space exploration. TARZAN supports shared integer variables (specified by using general arithmetic expressions), urgent locations, and invariants. The semantics of both invariants and networks in TARZAN follows those of Uppaal [7]. Support for networks of TA and integer variables in backward reachability is currently under development. Successors and predecessors algorithms were revised to allow each clock to have its own maximum constant.

TARZAN accepts .txt input files, parsed using the Boost Spirit X3 library, containing a TA representation written in a domain-specific language called *Liana*. Each file represents a single TA; a network of TA is defined using multiple files.

Networks of TA A *network* of size n is the parallel composition of n individual TA [4, Chapter 9]. Given n regions $\mathcal{R}_1, \dots, \mathcal{R}_n$, each corresponding to a TA $\mathcal{A}_1, \dots, \mathcal{A}_n$ in the network, a *network region* $\overline{\mathcal{R}}$ is defined as $\overline{\mathcal{R}} := \{\mathcal{R}_1, \dots, \mathcal{R}_n\}$. To compute both immediate delay and discrete successors of $\overline{\mathcal{R}}$, TARZAN relies on specialized data structures that generalize the applicability of the algorithms originally developed for individual regions to network regions. These data structures ensure the correctness of on-the-fly immediate delay and discrete successors computation across the Cartesian product of $\mathcal{A}_1, \dots, \mathcal{A}_n$. In particular, a C++ double-ended queue maintains the relative ordering of the fractional parts of all clocks within $\overline{\mathcal{R}}$, partitioning these clocks into ordered equivalence classes. This ordering captures the evolution over time of all clocks within $\overline{\mathcal{R}}$, including which clocks will exit the unit (possibly becoming unbounded), reach the next unit, or be reset. Furthermore, a C++ set identifies the regions in $\overline{\mathcal{R}}$ that are of class Z or M . Since the generalized algorithms for network regions operate by applying the

successors algorithms to a subset of individual regions in $\overline{\mathcal{R}}$, this classification is essential for computing immediate delay successors; indeed, regions of class Z or M contain clocks that will exit the unit; hence, their immediate delay successors must be computed prior to those of regions of class P .

Implementation considerations Since the focus of this paper is the implementation of algorithms using the new representation of Equation (1), most optimizations are deferred to future work. Currently, TARZAN does not implement any optimizations for state space exploration, except for symmetry reduction [8]. Memory compression optimizations are not implemented either. The algorithms also suffer when computing delay successors (predecessors), as these computations require generating a sequence of immediate delay successors (predecessors). This limitation could be mitigated by implementing the optimization described in Section 4 for delay predecessors (which can be adapted for delay successors as well). Furthermore, since TARZAN only performs reachability analysis, proving that a region is unreachable going forward requires exploring the entire reachable state space. Due to the aforementioned limitations, a BFS exploration must visit all delay successors (predecessors) before visiting discrete successors (predecessors), which is typically unnecessary and increases the verification time. For this reason, unless stated otherwise, all experiments follow a DFS strategy.

5.2 Forward reachability benchmarks

We compared TARZAN against Uppaal 5.0 and TChecker (commit d711ace) in forward reachability¹. The benchmarks (adjusted to ensure compatibility across all tools) come from publicly available repositories, published literature, or manually crafted when required. If possible, the reachability queries match those of the original benchmarks; otherwise, they have been adapted to fit the capabilities of the tools. Uppaal was run with standard DBM without memory optimizations, TChecker was configured with the `covreach` flag to match the algorithm used in Uppaal. Experiments were conducted on a MacBook Pro equipped with an M3 chip and 24 GB of RAM.

In the following, tables report Verification Time (VT), Execution Time (ET), memory (Mem, representing the Maximum Resident Set Size), and the number of stored regions and states. ET measures the time elapsed between issuing a verification command and receiving the result from a tool. Timings are in seconds, memory in MB. The size of a network is denoted by K (for **flower**, K indicates the number of clocks). All values are averages over five runs, with a timeout of 600 s set on ET. Timeouts are reported as TO, ε denotes times below 0.001 s, and out-of-memory events are denoted by OOM. Bold cells indicate the

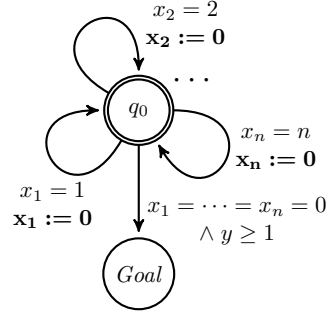


Fig. 2: **flower**. q_0 is initial.

¹ To ensure a fair comparison, symmetry reduction was disabled in both TARZAN and Uppaal; notably, this optimization cannot be manually controlled in TChecker.

Table 1: When all transitions have punctual guards, regions may outperform zones. Here, **flower** is a single TA, while the others are networks of TA.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► boolean [26] :: [Query: $\exists \Diamond(ctr_1 = 1 \wedge ctr_2 = 1 \wedge \dots \wedge ctr_K = 1)$] ◀												
4	ϵ	0.013	7.78	24	ϵ	0.007	6.47	185	0.001	0.052	18.85	185
6	0.003	0.016	13.42	425	0.088	0.096	13.80	8502	0.075	0.128	19.91	8502
8	0.013	0.027	29.10	1009	75.461	75.980	220.82	566763	77.204	77.523	81.79	566763
► flower [21] :: [Query: $\exists \Diamond(Flowe\text{r}.Goal)$] ◀												
5	ϵ	0.013	7.92	95	0.001	0.007	6.35	150	0.001	0.056	18.82	150
7	ϵ	0.022	10.65	573	0.322	0.330	13.12	3508	0.399	0.455	19.76	3508
9	0.007	0.022	58.62	9161	—	TO	—	—	—	TO	—	—
► gates [26] :: [Query: $\exists \Diamond(Unlocker.Goal)$] ◀												
5	ϵ	0.013	8.84	76	0.001	0.008	7.19	549	0.001	0.055	19.01	549
7	ϵ	0.014	11.23	151	0.110	0.122	21.08	32979	0.035	0.092	22.79	32979
9	0.001	0.015	15.43	251	47.895	48.771	806.72	3071646	5.228	6.509	451.71	3071646
► ring [26] :: [Query: $\exists \Diamond(P_0.Goal \wedge P_1.Goal \wedge \dots \wedge P_K.Goal)$] ◀												
4	0.001	0.015	12.40	542	0.020	0.029	11.56	7738	0.010	0.065	19.52	7738
6	0.011	0.025	48.96	3397	12.137	12.561	268.93	1180575	3.406	3.715	128.87	1180575
8	0.204	0.225	687.77	47063	—	TO	—	—	—	TO	—	—

fastest VT (on ties, the fastest ET). KO indicates that an error occurred during verification. The complete benchmark suite is in the Appendix, Section E.

TA with punctual guards Region-based abstractions can outperform zone-based approaches when TA are *closed*, *i.e.*, contain only non-strict guards. This is particularly evident when guards are *punctual*, *i.e.*, restricted to equality constraints. The **flower** TA [21] depicted in Figure 2, which is also used in our evaluation of backward reachability, satisfies these requirements. It consists of two locations: q_0 (which is initial) and $Goal$, and it features $n+1$ clocks, denoted by $x_1, \dots, x_n, y$. The guards over clocks x_i are punctual; the guard on y is non-strict. Since y is never reset, the behavior of **flower** is equivalent to a model featuring only punctual guards. To reach $Goal$, all clocks x_i must simultaneously be exactly zero, while y must satisfy $y \geq 1$. Specifically, when transitioning to $Goal$, y must be equal to the least common multiple of $1, 2, \dots, n$. Similar behavior is shown in the additional examples reported in Table 1: these are networks of TA with only punctual guards. In all these experiments, punctual guards force zones to be repeatedly and finely split, as reflected by the large number of states shown in Table 1. The results indicate that Uppaal and TChecker experience a substantial decrease in performance when $K \geq 8$.

Closed TA Two networks of closed TA were adapted from the benchmarks available on [32]: **medicalWorkflow** models medical checks for patients, while **mpeg2** models the homonymous compression standard, where compression leverages a given number of bidirectional frames (BFrames). Since TARZAN does not currently support broadcast channels, these models were rewritten into equivalent versions that use only point-to-point channels. The results in Table 2 reveal a high number of stored states. This indicates that these models restrict the evolution of clocks in a way that forces zones to be repeatedly and finely split, thus pushing these models into a category where zone-based abstractions exhibit poor

Table 2: The results are derived from networks of closed TA, where guards may either be punctual or contain non-strict inequalities. In this setting, TARZAN can handle both large networks (`medicalWorkflow`) and large constants (`mpeg2`).

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► <code>medicalWorkflow</code> [32] :: [Query: $\exists\Diamond(Patient_1.Done \wedge Patient_2.Done \wedge \dots \wedge Patient_{K-2}.Done)$] ◀												
12	0.007	0.021	33.39	660	0.011	0.030	22.63	639	0.002	0.059	20.83	582
22	0.056	0.072	204.97	2715	0.316	0.560	173.75	3774	0.032	0.093	31.76	2862
42	0.609	0.647	2271.65	15025	13.571	16.096	2517.18	25544	1.275	1.379	230.07	16522
72	5.781	5.959	12891.17	68990	355.827	389.223	16109.45	126949	54.200	54.838	2799.57	74762
102	27.918	28.466	13780.57	189555	KO	KO	KO	KO	593.317	597.306	12364.67	202302
► <code>mpeg2</code> [32] :: [Query: $\exists\Diamond(BFrame_1.Bout \wedge BFrame_2.Bout \wedge \dots \wedge BFrame_{K-4}.Bout)$] ◀												
8	0.365	0.385	451.46	26916	0.003	0.014	8.96	531	0.001	0.058	19.46	537
12	0.796	0.819	718.63	27238	8.405	8.663	252.70	383103	1.621	1.766	112.33	383109
16	1.364	1.392	1032.46	27704	—	TO	—	—	—	TO	—	—

Table 3: Large constants negatively impact the VT of regions, unlike zones. Here, the first row of `pagerank` corresponds to the variant with truncated constants.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► <code>CSMA/CD</code> [23] :: [Query: $\exists\Diamond\left(P_1.senderRetry \wedge P_2.senderRetry \wedge P_3.senderTransm \wedge P_4.senderRetry \wedge \dots \wedge P_7.senderRetry\right)$] ◀												
21	0.198	0.215	364.96	20936	0.004	0.038	30.32	232	0.001	0.061	20.24	269
31	0.674	0.701	1194.45	47526	0.018	0.218	117.03	371	0.002	0.064	21.11	554
51	3.364	3.431	5351.35	131906	0.293	1.386	837.12	1381	0.010	0.077	23.89	1424
► <code>pagerank</code> [5] :: [Query: $\exists\Diamond(Stage_7.Completed)$] ◀												
9	0.068	0.085	179.33	4193	0.002	0.086	58.54	1129	0.002	0.076	22.89	1129
9	4.227	4.361	10950.36	257311	0.002	0.085	58.53	1129	0.002	0.076	23.01	1129

performance, a phenomenon already demonstrated in the above experiments (see Table 1). Furthermore, both Uppaal and TChecker exhibit an unexpectedly high verification time in `medicalWorkflow`. This indicates that the dominant cost in this benchmark originates from zone manipulation, whose operations may have quadratic or cubic complexity in the number of clocks [21]. Region-based analysis avoids this overhead, as its fundamental operations have linear complexity in the number of clocks. Interestingly, TChecker aborted execution on the largest instances of `medicalWorkflow` raising the error: *"ERROR: value out of bounds"*.

Large constants When constants are large, zones are still preferable. Table 3 reports results from two real-world case studies: the Carrier Sense Multiple Access / Collision Detection protocol (`CSMA/CD`) [23] and `pagerank` [5], a mechanism used to prioritize Web keyword search results. The latter is presented in two variants: in the first, the last two digits of the original constants were truncated, while the second uses the full constants. The largest constant in `CSMA/CD` is 808, whereas in `pagerank` they are 101 (truncated) and 10162 (full).

Safety properties When safety properties must be verified using forward reachability, TARZAN requires to explore the entire reachable state space. Table 4 reports the results for the Fischer mutual exclusion protocol [34]. The verified property specifies that two given processes must never be in the critical section simultaneously. In the reminder of this section, we demonstrate that backward

Table 4: Safety properties require TARZAN to visit the entire reachable state space.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► fischer [34] :: [Query: $\exists \Diamond (Fischer_1.cs \wedge Fischer_2.cs)$] ◀												
4	0.644	0.669	1098.36	180032	ϵ	0.007	6.75	220	0.001	0.055	18.96	220
5	19.627	19.849	12948.99	3920652	0.004	0.011	9.30	727	0.002	0.057	19.07	727
6	—	—	OOM	—	0.024	0.032	12.25	2378	0.008	0.064	19.43	2378

reachability may enhance the verification of safety properties by exploring only a restricted subset of the state space, potentially avoiding out-of-memory events.

5.3 Backward reachability analysis with TARZAN

We begin with a backward unreachability example on **flower**, and then compare its forward and backward reachability results. The entire **flower** reachable state space, when considering five clocks and computed going forward starting from the initial region $\mathcal{R}_0 = \{q_0, h(x_1) = h(x_2) = h(x_3) = h(x_4) = h(y) = 0, X_0\}$, where $X_{\mathcal{R}_0,0} = \{x_1, x_2, x_3, x_4, y\}$, resulted in 1517 regions, computed in 1.18 ms.

Backward reachability can offer a practical advantage for proving unreachability, since it may avoid a complete state space exploration. Consider the region $\mathcal{R}_1 = \{q_0, h(x_1) = 1, h(x_2) = 2, h(x_3) = h(x_4) = 0, h(y) = 1, X_{-1}, X_0\}$, where $X_{\mathcal{R}_1,-1} = \{x_2, y\}$ and $X_{\mathcal{R}_1,0} = \{x_1, x_3, x_4\}$. $\mathcal{R}_1$ is unreachable: it requires both x_2 and y to become unbounded simultaneously, but the constraint $y \geq 1$ forces y to become unbounded before x_2 . TARZAN classifies $\mathcal{R}_1$ as unreachable by verifying that $\mathcal{R}_0$ is unreachable from $\mathcal{R}_1$, requiring 272 regions in 0.245 ms. By adjusting $\mathcal{R}_1$ with the correct ordering in which clocks become unbounded, we obtain $\mathcal{R}_2 = \{q_0, h(x_1) = 1, h(x_2) = 2, h(x_3) = h(x_4) = 0, h(y) = 1, X_{-2}, X_{-1}, X_0\}$, where $X_{\mathcal{R}_2,-2} = \{x_2\}$, $X_{\mathcal{R}_2,-1} = \{y\}$, and $X_{\mathcal{R}_2,0} = \{x_1, x_3, x_4\}$. Region $\mathcal{R}_2$ is reachable. Reaching $\mathcal{R}_0$ from $\mathcal{R}_2$ required 263 regions in 0.378 ms.

Now, consider the version of **flower** with $K = n + 1$ clocks. Let $\mathcal{R}_K$ denote the region in location *Goal* in which all clocks x_i are exactly equal to zero (no fractional part) and $y \geq 1$ holds. Table 5 reports the VT and number of regions stored by TARZAN when checking the forward and backward reachability of $\mathcal{R}_K$. The results show that forward reachability is generally faster and requires fewer regions, while backward reachability remains comparable in this setting.

Backward reachability was employed to verify that in **fischer** two processes cannot occupy the critical section simultaneously. As backward reachability does not currently support networks of TA nor integer variables, we developed a script to construct a single TA version of **fischer** [26], in which the *pid* variable is directly encoded within its locations. The safety property to be verified with K processes is described as follows: processes 1 through $K - 2$ are requesting access to the critical section, with their clocks strictly between 1 and 2; processes $K - 1$ and K are both in the critical section, with their clocks greater than 2; *pid* is equal to K . A preliminary phase was necessary to compute the initial subset of regions from which backward reachability was started. This required consider-

Table 5: Backward reachability results. **fischer** goes out of memory at $K = 6$. Tests use DFS except **trainAHV93**, for which BFS is faster. **flower** checks reachability, **fischer** unreachable, and **trainAHV93** reachability (rows 1,2) and unreachable (rows 3,4).

► flower [21] :: DFS ◀				► fischer [26] :: DFS ◀				► trainAHV93 [26] :: BFS ◀			
<i>Forward</i>		<i>Backward</i>		<i>Forward</i>		<i>Backward</i>		<i>Forward</i>		<i>Backward</i>	
$K = 11, 13, 15, 17$		$K = 11, 13, 15, 17$		$K = 3, 4, 5, 6$		$K = 3, 4, 5, 6$		$K = 4, 5, 4, 5$		$K = 4, 5, 4, 5$	
VT	Regions	VT	Regions	VT	Regions	VT	Regions	VT	Regions	VT	Regions
0.028	30761	0.060	38669	0.007	9083	ϵ	264	ϵ	1241	0.171	118868
0.412	370331	0.615	388911	0.141	183459	0.001	1316	0.026	31516	66.877	31051667
7.188	5142671	9.414	4921043	4.061	4182681	0.008	8724	0.013	17124	0.030	26914
16.212	11023829	21.497	11077295	—	—	0.100	77084	1.000	912597	0.635	614220

ing all ordered partitions of the fractional parts of bounded clocks as well as all ordered partitions of unbounded clocks. Clock constraints in the property were introduced to enable the manual computation of this subset. Backward reachability results, reported in Table 5, demonstrate superior performance compared to forward reachability, to the extent that an out-of-memory event becomes feasible. Indeed, the unreachable portion of the state space limited the backward exploration to a relatively small number of regions. We evaluated the same property with Uppaal (which uses forward reachability) on the same **fischer** model and obtained: for $K = 3$, 88 states in 0.001 s; for $K = 4$, 460 states in 0.004 s; for $K = 5$, 2765 states in 0.025 s; and for $K = 6$, 19042 states in 0.214 s. In this comparison, TARZAN outperformed Uppaal when using backward reachability.

Backward reachability was also applied to **trainAHV93** [26] (we developed a script to convert it into a single TA version). Exploration was conducted in BFS, as it achieved better performance on this model. **trainAHV93** allows two or more trains to be in the crossing while the gate is down. Rows 1,2 of Table 5 report the results for (i) reaching the first region detected by the BFS exploration in which at least two trains occupy the crossing while the gate is down, and (ii) computing the backward predecessors from this region to an initial one. In this case, backward reachability requires substantially more regions than forward reachability. We also verified the safety property stating that whenever a train is in the crossing, the number of trains in the crossing cannot be zero (*i.e.*, no counter malfunction). Such property with $K - 2$ trains, a gate, and a controller, is described as follows: trains 1 to $K - 3$ are not in the crossing and their clocks are strictly between 2 and 3; train $K - 2$ is in the crossing and its clock is exactly 4; the gate’s clock is strictly between 1 and 2, and the controller’s clock is greater than 1. As in **fischer**, we first explicitly identified the initial subset of regions from which backward reachability was started. Rows 3,4 of Table 5 show that, in this case, forward and backward reachability perform comparably.

6 Conclusion and Future Works

We introduced TARZAN, a region-based verification library supporting forward and backward reachability for TA. Our experiments demonstrated that TARZAN

exhibits superior performance on closed TA and TA with punctual guards, while zone-based approaches remain better suited to other classes of TA. We showed that backward reachability with TARZAN is often feasible in practice and effective for verifying safety properties. For this reason, the integration of TARZAN into existing tools has the potential to enhance their verification capabilities.

Future work will proceed in several directions. We plan to integrate additional optimizations into TARZAN, with a particular focus on improving the computation of delay successors and predecessors, while leveraging symbolic techniques to reduce memory usage. In addition, we will extend backward reachability algorithms to networks of TA, thereby laying the foundation for applying TARZAN to the synthesis of winning strategies in Timed Games. Finally, we intend to explore hybrid approaches that complement TARZAN with zone-based techniques, thereby combining the strengths of both abstractions.

References

1. Alur, R., Dill, D.L.: A theory of timed automata. *Theoretical Computer Science* **126**(2), 183–235 (1994)
2. André, E., Jacobs, S., Karra, S.L., Sankur, O.: Parameterized verification of timed networks with clock invariants (2024)
3. Apron developers: Apron. <https://github.com/antoinemine/apron>, accessed: 2025-11-24
4. Baier, C., Katoen, J.P.: Principles of Model Checking (Representation and Mind Series). The MIT Press (2008)
5. Baresi, L., Bersani, M.M., Marconi, F., Quattrocchi, G., Rossi, M.: Using formal verification to evaluate the execution time of spark applications. *Formal Aspects of Computing* **32**(1), 33–70 (2020). <https://doi.org/10.1007/s00165-020-00505-4>, <https://doi.org/10.1007/s00165-020-00505-4>
6. Behrmann, G., David, A., Larsen, K.G.: A Tutorial on Uppaal, pp. 200–236. Springer Berlin Heidelberg, Berlin, Heidelberg (2004)
7. Bengtsson, J., Yi, W.: Timed Automata: Semantics, Algorithms and Tools, pp. 87–124. Springer Berlin Heidelberg, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-27755-2_3, https://doi.org/10.1007/978-3-540-27755-2_3
8. Bošnački, D., Dams, D., Holenderski, L.: Symmetric spin. In: Havelund, K., Penix, J., Visser, W. (eds.) SPIN Model Checking and Software Verification. pp. 1–19. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)
9. Bouyer, P.: Forward analysis of updatable timed automata. *Formal Methods in System Design* **24**(3), 281–320 (May 2004)
10. Bouyer, P., Brinksma, E., Larsen, K.G.: Optimal infinite scheduling for multi-priced timed automata. *Formal Methods in System Design* **32**(1), 3–23 (February 2008)
11. Bouyer, P., Fahrenberg, U., Larsen, K.G., Markey, N.: Quantitative analysis of real-time systems using priced timed automata. *Commun. ACM* **54**(9), 78–87 (Sep 2011)
12. Bouzenada, A., Saidouni, D.E., Díaz, G.: A finite representation of durational action timed automata semantics. *Mathematics* **12**(24) (2024)
13. Bozga, M., Daws, C., Maler, O., Olivero, A., Tripakis, S., Yovine, S.: Kronos: A model-checking tool for real-time systems. In: Ravn, A.P., Rischel, H. (eds.) Formal

- Techniques in Real-Time and Fault-Tolerant Systems. pp. 298–302. Springer Berlin Heidelberg, Berlin, Heidelberg (1998)
14. Comtet, L.: Advanced Combinatorics: The Art of Finite and Infinite Expansions. Springer Dordrecht, Dordrecht, Netherlands, 1 edn. (2012), reprint of the 1974 original edition by D. Reidel Publishing Company
 15. Daszczuk, W.B.: Modeling and verification of asynchronous systems using timed integrated model of distributed systems. *Sensors* **22**(3) (2022). <https://doi.org/10.3390/s22031157>, <https://www.mdpi.com/1424-8220/22/3/1157>
 16. Dill, D.L.: Timing assumptions and verification of finite-state concurrent systems. In: Sifakis, J. (ed.) *Automatic Verification Methods for Finite State Systems*. pp. 197–212. Springer Berlin Heidelberg, Berlin, Heidelberg (1990)
 17. Farkas, R., Bergmann, G.: Towards reliable benchmarks of timed automata. In: Pataki, B. (ed.) *Proceedings of the 25th PhD Mini-Symposium*. pp. 20–23. Budapest University of Technology and Economics, Department of Measurement and Information Systems (2018)
 18. Henzinger, T., Nicollin, X., Sifakis, J., Yovine, S.: Symbolic model checking for real-time systems. *Information and Computation* **111**(2), 193–244 (1994)
 19. Herbreteau, F., Srivathsan, B., Tran, T.T., Walukiewicz, I.: Why liveness for timed automata is hard, and what we can do about it. *ACM Trans. Comput. Logic* **21**(3) (2020)
 20. Jensen, N.Ø., Jensen, P.G., Larsen, K.G.: Dynamic extrapolation in extended timed automata. In: Li, Y., Tahar, S. (eds.) *Formal Methods and Software Engineering (ICFEM 2023)*. *Lecture Notes in Computer Science*, vol. 14308. Springer, Singapore (2023). https://doi.org/https://doi.org/10.1007/978-981-99-7584-6_6
 21. Jørgensen, K.Y., Larsen, K.G., Srba, J.: Time-darts: A data structure for verification of closed timed automata. *Electronic Proceedings in Theoretical Computer Science* **102**, 141–155 (Nov 2012)
 22. Kereskényi-Balogh, Z., Nyul, G.: Fubini numbers and polynomials of graphs. *Mediterranean Journal of Mathematics* **18**(6), 230 (2021)
 23. Kiviriga, A., Larsen, K.G., Nyman, U.: Randomized reachability analysis in uppaal: Fast error detection in timed systems. In: Lluch Lafuente, A., Mavridou, A. (eds.) *Formal Methods for Industrial Critical Systems (FMICS 2021)*. *Lecture Notes in Computer Science*, vol. 12863. Springer, Cham (2021). https://doi.org/https://doi.org/10.1007/978-3-030-85248-1_9
 24. Lehmann, S., Schupp, S.: Bounded dbm-based clock state construction for timed automata in uppaal. *International Journal on Software Tools for Technology Transfer* **25**(1), 19–47 (February 2023)
 25. Maler, O., Pnueli, A., Sifakis, J.: On the synthesis of discrete controllers for timed systems. In: Mayr, E.W., Puech, C. (eds.) *STACS 95*. pp. 229–242. Springer Berlin Heidelberg, Berlin, Heidelberg (1995)
 26. Manini, A.: Tarzan: Timed automata region and zone library for real-time systems analysis. <https://github.com/andreamanini98/TARZAN> (2025)
 27. Manini, A., Rossi, M., San Pietro, P.: TARZAN Artifact (Feb 2026). <https://doi.org/10.5281/zenodo.18656202>, <https://doi.org/10.5281/zenodo.18656202>
 28. Peter, H.J., Ehlers, R., Mattmüller, R.: Synthia: Verification and synthesis for timed automata. In: Gopalakrishnan, G., Qadeer, S. (eds.) *Computer Aided Verification*. pp. 649–655. Springer Berlin Heidelberg, Berlin, Heidelberg (2011)
 29. PPL developers: Ppl: Parma polyhedra library. <https://github.com/BUGSENG/PPL>, accessed: 2025-11-24

30. PyDBM developers: Pydbm. <https://launchpad.net/pydbm>, accessed: 2025-11-24
31. Rodriguez-Navas, G., Proenza, J.: Using timed automata for modeling distributed systems with clocks: Challenges and solutions. *IEEE Transactions on Software Engineering* **39**(6), 857–868 (2013). <https://doi.org/10.1109/TSE.2012.73>
32. TAPAAL Developers: Tapaal: Timed-arc petri net analyzer. <https://www.tapaal.net/download/>, accessed: 2025-11-15
33. UDBM developers: Udbm: Uppaal dbm library. <https://github.com/UPPAALModelChecker/UDBM>, accessed: 2025-11-24
34. UPPAAL Developers: Uppaal. <https://uppaal.org/>, accessed: 2025-11-15
35. Wang, F.: Efficient verification of timed automata with bdd-like data structures. *International Journal on Software Tools for Technology Transfer* **6**(1), 77–97 (July 2004)
36. Yovine, S.: Model checking timed automata. In: Rozenberg, G., Vaandrager, F.W. (eds.) *Lectures on Embedded Systems: European Educational Forum School on Embedded Systems Veldhoven, The Netherlands November 25–29, 1996*, pp. 114–152. Springer Berlin Heidelberg, Berlin, Heidelberg (1998)
37. Yu, F., Wang, B.Y.: Toward unbounded model checking for region automata. In: Wang, F. (ed.) *Automated Technology for Verification and Analysis*. pp. 20–33. Springer Berlin Heidelberg, Berlin, Heidelberg (2004)

A Immediate Delay Successors

Algorithm 3, whose time complexity is $O(|X_{\mathcal{A}}|)$, computes the (unique) immediate delay successor of a given region $\mathcal{R}$, based on the class to which $\mathcal{R}$ belongs. In particular, Line 2 creates a copy $\mathbf{R}$ of $\mathcal{R}$, returned as the result in Line 21. If $\mathcal{R}$ belongs to class U (Line 3), it has no delay successors at all, so we return the same region on Line 4. If $\mathcal{R}$ belongs to either class Z or class M (Line 5), two auxiliary sets are initialized on Line 6: X_{tmp} stores the clocks that exit the unit (*i.e.*, leaving $X_{\mathbf{R},0}$), while X_{oob} collects the clocks that become unbounded. For each clock $x \in X_{\mathbf{R},0}$ (with no fractional part), the loop at Line 7 adds x to either X_{oob} (Line 9) or X_{tmp} (Line 11), and removes x from $X_{\mathbf{R},0}$ (Line 12). If any clocks left the unit, they are inserted to the right of $X_{\mathbf{R},0}$, as they now have the smallest fractional part (Lines 13,14). If a clock becomes unbounded, it is placed to the left of $X_{\mathbf{R},-\ell_{\mathbf{R}}}$, reflecting that it is the most recently unbounded clock (Lines 15,16). If $\mathcal{R}$ is of class P (Line 17), then the clocks with the highest fractional part (*i.e.*, those in $X_{\mathbf{R},\tau_{\mathbf{R}}}$) reach the next unit. The loop at Line 18 increments their integer value by 1 (Line 19). Since these clocks have entered the next unit, they are inserted into $X_{\mathbf{R},0}$, and $X_{\mathbf{R},\tau_{\mathbf{R}}}$ is removed from $\mathbf{R}$ (Line 20).

B Discrete predecessors

The discrete predecessors of a region $\mathcal{R}$ can be computed using Algorithm 4. In particular, Line 2 initializes a set storing the computed predecessor regions. A loop over all transitions t ending in $q_{\mathcal{R}}$, the location of $\mathcal{R}$, begins at Line 3. Line 4 initializes a set X_{bnd} , which will contain clocks that were bounded or possibly unbounded, and a set X_{oob} , which will contain clocks that were certainly unbounded (in a predecessor). A copy $\mathbf{R}$ of $\mathcal{R}$ is made at Line 5. Line 6 creates a reference $\mathbf{X}_{\mathbf{R}}$ to the clock sets of $\mathbf{R}$, meaning that changes to $\mathbf{X}_{\mathbf{R}}$ (*e.g.*, removing a clock) directly affect $\mathbf{R}$. The location of $\mathbf{R}$ is updated on Line 7 to match the one where t originates. Then, the loop at Line 8 examines each reset clock $x \in Y$: (i) if x appears in an equality clock constraint of the form $(x = c) \in \gamma$, only the integer value of x must be set (Lines 9,10); (ii) if x was unbounded, *i.e.*, it appears in an inequality constraint of the form $(x > c_m) \in \gamma$, x is added to X_{oob} and removed from $\mathbf{R}$ (Lines 11,13), while its integer value is set to c_m (Line 12); (iii) otherwise, x was bounded or possibly unbounded, hence x is added to X_{bnd} and removed from $\mathbf{R}$ (Lines 14,15). The latter case also accounts for reset clocks $x' \in Y$ that do not appear in any constraint of γ , which is equivalent to assuming a trivial constraint of the form $(x' \geq 0) \in \gamma$. It is important to note that every clock in Y is required to be exactly zero, *i.e.*, to have an integer value equal to zero and no fractional part, since all clocks in Y are reset along transition t .

If X_{bnd} is empty (Line 16), the predecessors over the current transition are computed by finding all ordered partitions of X_{oob} , preserving the order of unreset unbounded clocks. This is performed at Line 17 using the auxiliary algorithm $part\text{-}regs(\mathcal{R}', i, j, X, \mathcal{H})$, which computes all regions derived from the input region $\mathcal{R}'$ by finding all ordered partitions of a clock set X , while preserving the

Algorithm 3: find-immediate-delay-successor($\mathcal{R}$)

Data: $\mathcal{R}$: a region.
Result: The immediate delay successor of $\mathcal{R}$.

```

1 begin
2    $\mathbf{R} \leftarrow \text{copy of } \mathcal{R};$ 
3   if  $X_{\mathcal{R},0} = \emptyset \wedge \mathbf{r}_{\mathcal{R}} = 0$  then
4     //  $\mathcal{R}$  belongs to class U
5     return  $\mathcal{R};$ 
6   else if  $X_{\mathcal{R},0} \neq \emptyset$  then
7     //  $\mathcal{R}$  belongs to either class Z or class M
8      $X_{tmp} \leftarrow \emptyset; X_{oob} \leftarrow \emptyset;$ 
9     foreach  $x \in X_{\mathcal{R},0}$  do
10      if  $h_{\mathbf{R}}(x) = c_{\max}$  then
11         $X_{oob} \leftarrow X_{oob} \cup \{x\};$ 
12      else
13         $X_{tmp} \leftarrow X_{tmp} \cup \{x\};$ 
14       $X_{\mathcal{R},0} \leftarrow X_{\mathcal{R},0} \setminus \{x\};$ 
15    if  $X_{tmp} \neq \emptyset$  then
16       $\text{insert } X_{tmp} \text{ to the right of } X_{\mathcal{R},0};$ 
17    if  $X_{oob} \neq \emptyset$  then
18       $\text{insert } X_{oob} \text{ to the left of } X_{\mathcal{R},-\ell_{\mathbf{R}}};$ 
19  else
20    //  $\mathcal{R}$  belongs to class P
21    foreach  $x \in X_{\mathcal{R},\mathbf{r}_{\mathbf{R}}}$  do
22       $h_{\mathbf{R}}(x) \leftarrow h_{\mathbf{R}}(x) + 1;$ 
23     $\text{insert clocks of } X_{\mathcal{R},\mathbf{r}_{\mathbf{R}}} \text{ in } X_{\mathcal{R},0} \text{ and remove } X_{\mathcal{R},\mathbf{r}_{\mathbf{R}}} \text{ from } \mathbf{R};$ 
24  return  $\mathbf{R};$ 

```

order of clock sets $X_{\mathcal{R}',i}, \dots, X_{\mathcal{R}',j}$ (where $i \leq j$). For each resulting region, the integer value of every clock x is given by $\mathcal{H}(x)$. If $X = \emptyset$, *part-regs* returns the input region $\mathcal{R}'$ unchanged. This also covers the case where a transition does not reset any clock; in such a situation, both X_{bnd} and X_{oob} are empty, and Line 17 simply inserts $\mathbf{R}$ into the result set. The pseudocode of *part-regs* is given in Algorithm 5, and a detailed description follows at the end of this section.

Otherwise, all combinations of integer values (and potential unboundedness) for clocks in X_{bnd} must be considered. This is done in the loop at Line 19, which iterates over all functions $\bar{h}$ defined on X_{bnd} . Note that $\bar{h}$ maps only clocks in X_{bnd} , while $h_{\mathbf{R}}$ maps clocks in $\mathbf{X}_{\mathbf{R}} \setminus X_{bnd}$; the conventional value $c_m + 1$ captures the case in which a clock was unbounded in the predecessor. All clocks in X_{bnd} that are unbounded under $\bar{h}$ (i.e., the integer value of these clocks returned by $\bar{h}$ is $c_m + 1$) are collected into a set Δ (Line 20). The loop at Line 21 iterates over all regions generated by *part-regs* from $\mathbf{R}$ partitioning $X_{oob} \cup \Delta$, preserving the order of unreset unbounded clocks. For each such region R , *part-regs* is applied

Algorithm 4: find-discrete-predecessors($\mathcal{R}$)

Data: $\mathcal{R}$: a region.
Result: The discrete predecessors of $\mathcal{R}$.

```

1 begin
2    $res \leftarrow \emptyset$ ;
3   foreach  $t = (q, a, \gamma, Y, q_{\mathcal{R}}) \in T$  do
4      $X_{bnd} \leftarrow \emptyset$ ;  $X_{oob} \leftarrow \emptyset$ ;
5      $\mathbf{R} \leftarrow \text{copy of } \mathcal{R}$ ;
6      $\mathbf{X}_{\mathbf{R}} \leftarrow \text{a reference to the clocks of } \mathbf{R}$ ;
7      $q_{\mathbf{R}} \leftarrow q$ ;
8     foreach clock  $x \in Y$  do
9       if  $x$  belongs to a constraint of the form  $(x = c) \in \gamma$  then
10          $h_{\mathbf{R}}(x) = c$ ;
11       else if  $x$  belongs to a constraint of the form  $(x > c_m) \in \gamma$  then
12          $h_{\mathbf{R}}(x) = c_m$ ;
13          $X_{oob} \leftarrow X_{oob} \cup \{x\}$ ;  $\mathbf{X}_{\mathbf{R}} \leftarrow \mathbf{X}_{\mathbf{R}} \setminus \{x\}$ ;
14       else
15          $X_{bnd} \leftarrow X_{bnd} \cup \{x\}$ ;  $\mathbf{X}_{\mathbf{R}} \leftarrow \mathbf{X}_{\mathbf{R}} \setminus \{x\}$ ;
16     if  $X_{bnd} = \emptyset$  then
17        $res \leftarrow res \cup \text{part-regs}(\mathbf{R}, -\ell_{\mathbf{R}}, -1, X_{oob}, h_{\mathbf{R}})$ ;
18     else
19       foreach function  $\bar{h} : X_{bnd} \rightarrow \{0, \dots, c_m + 1\}$  do
20          $\Delta \leftarrow \{x \in X_{bnd} \mid \bar{h}(x) > c_m\}$ ;
21         foreach  $R \in \text{part-regs}(\mathbf{R}, -\ell_{\mathbf{R}}, -1, X_{oob} \cup \Delta, \bar{h} \cup h_{\mathbf{R}})$  do
22            $res \leftarrow res \cup \text{part-regs}(R, 0, \mathbf{r}_R, X_{bnd} \setminus \Delta, \bar{h} \cup h_R)$ ;
23   remove from  $res$  all regions not satisfying  $\gamma$ ;
24   return  $res$ ;

```

on Line 22 to R partitioning $X_{bnd} \setminus \Delta$ (i.e., the bounded clocks), preserving the order of unreset bounded clocks. On Line 23, all regions not satisfying the guard γ of t are removed from res . Line 24 returns the discrete predecessors of $\mathcal{R}$.

It is worth noting that the total number of functions $\bar{h}$ defined over X_{bnd} in Line 19 can be reduced by tracking the minimum and maximum integer values that the clocks may assume. For example, consider two clocks x, y appearing in the constraints $1 \leq x \wedge x \leq 4 \wedge y \geq 5$. In this case, it is unnecessary to generate all combinations of integer values for x and y from 0 to $c_m + 1$; instead, it suffices to generate combinations where $\bar{h}(x) \in [1, 4]$ and $\bar{h}(y) \in [5, c_m + 1]$. In addition, note that *part-regs* may generate invalid regions that must be discarded. This occurs, for instance, when a clock x appears in a constraint $x > c$ and is assigned the integer value $\bar{h}(x) = c$. In this case, x must have a fractional part greater than zero; however, when computing all possible ordered partitions, it may happen that x is assigned no fractional part. Similarly, if x appears in a constraint $x \leq c$ and is assigned $\bar{h}(x) = c$, it must have a fractional part equal to zero, but the

Algorithm 5: $\text{part-regs}(\mathcal{R}, i, j, X, \mathcal{H})$

Data: $\mathcal{R}$: a region; i, j : two integers such that $i \leq j$ holds; X : a set of clocks, $\mathcal{H}$: a function assigning integer values to clocks.

Result: all regions obtained from $\mathcal{R}$ by finding all ordered partitions of X while preserving the relative order of $X_{\mathcal{R},i}, \dots, X_{\mathcal{R},j}$.

```

1 begin
2    $res \leftarrow \emptyset$ ;
3    $\mathbf{R} \leftarrow \text{copy of } \mathcal{R}$ ;
4   if  $X = \emptyset$  then
5     return  $\{\mathcal{R}\}$ ;
6   foreach  $x \in X$  do
7     if  $i \geq 0 \wedge \mathcal{H}(x) = c_m$  then
8        $X_{\mathbf{R},0} \leftarrow X_{\mathbf{R},0} \cup \{x\}$ ;  $X \leftarrow X \setminus \{x\}$ ;
9    $\mathcal{X} \leftarrow \{X_{\mathbf{R},i}, \dots, X_{\mathbf{R},j}\}$ ;
10   $\Pi \leftarrow \text{all partitions of } X$ ;
11  foreach partition  $\pi \in \Pi$  do
12     $\mathcal{Z} \leftarrow \text{ordered partitions of } (\mathcal{X} \cup \pi) \text{ that preserve the relative order}$ 
      of the elements in  $\mathcal{X}$ ; these partitions are obtained by computing all
      permutations of  $\pi$  and interleaving them with  $\mathcal{X}$ , allowing elements
      of  $\pi$  to also be inserted within the elements of  $\mathcal{X}$ ;
13    foreach ordered partition  $\zeta \in \mathcal{Z}$  do
14       $R \leftarrow \text{copy of } \mathbf{R} \text{ such that } (\mathcal{X} \cup \pi) \text{ are partitioned according to } \zeta \text{ and}$ 
      clocks have a value given by  $\mathcal{H}$  (if  $\mathcal{H}(x) > c_m$ , then assign  $c_m$  to  $x$ );
15       $res \leftarrow res \cup \{R\}$ ;
16  return  $res$ ;
```

computation of ordered partitions may assign it a fractional part greater than zero. These incorrect edge cases can be handled by adding additional checks to Algorithm 4 and *part-regs*, which are omitted here for conciseness.

Description of *part-regs* Line 2 initializes the result set. Then, a copy $\mathbf{R}$ of $\mathcal{R}$ is made on Line 3 to avoid modifying the original input region. Notice that, if the set X is empty (Line 4), X has exactly one partition, *i.e.*, the empty set itself. Therefore, for convenience, the original region is returned on Line 5. The loop at Line 6 processes each clock $x \in X$, checking if it can be inserted into $X_{\mathbf{R},0}$ (Line 7) and doing so at Line 8, while also removing x from X . This is done since a bounded clock can have an integer value equal to c_m only when it has no fractional part. Lines 9 and 10 compute the sets for which the order must be preserved and the partitions Π of X , respectively.

The loop at Line 11 iterates over Π , generating ordered partitions preserving the order of $\mathcal{X}$ (*e.g.*, in a valid ordered partition involving $\mathcal{X} = \{X_1, X_2, X_3\}$, X_1 appears before X_2 , which appears before X_3). Ordered partitions are computed at Line 12; some of them may involve inserting elements of a given partition $\pi \in \Pi$ into elements of $\mathcal{X}$. For each valid ordered partition ζ (Line 13), a region

is created by copying $\mathbf{R}$ (Line 14), updating its sets of clocks according to ζ , and using $\mathcal{H}$ for assigning integer values to clocks (the values returned by $\mathcal{H}$ are clamped to ensure no value exceeds c_m). Each such region is added to the result set (Line 15), which is returned on Line 16.

Example 3. Let $\mathcal{A}$ be a TA such that $X_{\mathcal{A}} = \{x, y, w, p, z, s\}$ and $c_m = 5$, and let $t = (q_0, a, \gamma, Y, q) \in T$, where $\gamma := x \geq 0 \wedge y \geq 1 \wedge w > 5 \wedge p = 1 \wedge z > 3 \wedge s > 4$ and $Y = \{x, y, w, p\}$, be the transition of $\mathcal{A}$ over which, for illustrative purposes, we compute only one discrete predecessor, which we refer to as $\mathbf{R}$, of the following region: $\mathcal{R} = \{q, h(x) = h(y) = h(w) = h(p) = 0, h(z) = h(s) = 4, X_0, X_1, X_2\}$, where $X_0 = \{x, y, w, p\}$, $X_1 = \{z\}$, and $X_2 = \{s\}$. To better understand how Algorithm 4 works, we reference some of its lines in the following computation.

Since clocks z and s are not reset in t , they do not influence the computation of $\mathbf{R}$. From the analysis of Y (Lines 8 through 15), we can infer the following: x and y may have been either bounded or unbounded in $\mathbf{R}$ (here, we assume they were bounded), w was unbounded, and p had the value 1 with no fractional part. In particular, x and y are now collected in X_{bnd} . Since x and y are assumed to be bounded in $\mathbf{R}$, all possible combinations of their integer values must now be considered. For the purposes of this example, a single combination suffices. Let $\bar{h}$ (Line 19) be such that $\bar{h}(x) = 3$ and $\bar{h}(y) = 2$ (this allows to satisfy γ on Line 23, since $h_{\mathbf{R}}(w) = 5$, $h_{\mathbf{R}}(p) = 1$, and $h_{\mathbf{R}}(z) = h_{\mathbf{R}}(s) = 4$). Since the order of the fractional parts of x and y in $\mathbf{R}$ is unknown, all possible ordered partitions of X_{bnd} and X_{oob} must be explored, while preserving the order of the fractional parts of z and s (they are not reset in t). One such ordered partition can be generated using *part-regs* (Lines 21,22). Finally, $\mathbf{R} = \{q_0, h(x) = 3, h(y) = 2, h(w) = 5, h(p) = 1, h(z) = h(s) = 4, X_{-1}, X_0, X_1, X_2, X_3\}$, where $X_{-1} = \{w\}$, $X_0 = \{p\}$, $X_1 = \{z\}$, $X_2 = \{x, y\}$, and $X_3 = \{s\}$.

C Proof of Theorem 1

Theorem 1. *If the order in which clocks become unbounded is tracked, the number of immediate delay predecessors of a given region is at most three.*

Proof. We assume that all clocks in a given region have a value greater than zero, otherwise a delay predecessor cannot be computed. The proof is conducted by considering each class identified in Definition 4 and then analyzing all possible ways to transition backwards to predecessor regions using only delay transitions.

- *Class Z:* consider a region $\mathcal{R}_Z$ of class Z . The immediate delay predecessor regions that can be reached backwards from $\mathcal{R}_Z$ belong to class P . Hence, let $\mathcal{R}_P$ be a region of class P . Since $\tau_{\mathcal{R}_Z} = 0$, it follows that from $\mathcal{R}_P$ we can transition forward to $\mathcal{R}_Z$ only if $\tau_{\mathcal{R}_P} = 1$ holds (every clock must have a fractional part equal to zero when reaching $\mathcal{R}_Z$ from $\mathcal{R}_P$). Since the only way to transition backwards from $\mathcal{R}_Z$ to $\mathcal{R}_P$ is to remove all clocks from $X_{\mathcal{R}_Z,0}$ and insert them into $X_{\mathcal{R}_P,1}$, while also decreasing the integer values of those clocks by 1, it follows that a region of class Z has exactly one immediate delay predecessor region of class P .

- *Class P*: consider a region $\mathcal{R}_P$ of class P . The immediate delay predecessor regions that can be reached backwards from $\mathcal{R}_P$ belong to class Z or class M . Let $\mathcal{R}_Z$ be a region of class Z and $\mathcal{R}_M$ be a region of class M .
 1. *From $\mathcal{R}_P$ back to $\mathcal{R}_Z$* . This happens only if $\tau_{\mathcal{R}_P} = 1$. We must distinguish two further cases: (i) $\ell_{\mathcal{R}_P} = \ell_{\mathcal{R}_Z}$: in this case, $X_{\mathcal{R}_Z,0}$ contains the clocks that exited from the unit, thus entering in $X_{\mathcal{R}_P,1}$. Thus, to transition backwards from $\mathcal{R}_P$ to $\mathcal{R}_Z$, it suffices to remove all clocks from $X_{\mathcal{R}_P,1}$ and insert them into $X_{\mathcal{R}_Z,0}$; (ii) $\ell_{\mathcal{R}_P} = \ell_{\mathcal{R}_Z} + 1$: in this case, $X_{\mathcal{R}_Z,0}$ contains the clocks that exited from the unit and the clocks that became unbounded, i.e., $X_{\mathcal{R}_Z,0} = X_{\mathcal{R}_P,-\ell_{\mathcal{R}_P}} \cup X_{\mathcal{R}_P,1}$. Thus, to transition backwards from $\mathcal{R}_P$ to $\mathcal{R}_Z$, it suffices to remove all clocks from $X_{\mathcal{R}_P,-\ell_{\mathcal{R}_P}}$ and $X_{\mathcal{R}_P,1}$ and insert them into $X_{\mathcal{R}_Z,0}$. Hence, a region of class P has at most two immediate delay predecessor regions of class Z .
 2. *From $\mathcal{R}_P$ back to $\mathcal{R}_M$* . If $\tau_{\mathcal{R}_P} = 1$ holds, there is only one possible way to transition forward from $\mathcal{R}_M$ to $\mathcal{R}_P$, i.e., when $\tau_{\mathcal{R}_M} = 1$ and all clocks in $X_{\mathcal{R}_M,0}$ must become unbounded. Hence, it is possible to transition backwards from $\mathcal{R}_P$ to $\mathcal{R}_M$ by removing all clocks from $X_{\mathcal{R}_P,-\ell_{\mathcal{R}_P}}$ and inserting them into $X_{\mathcal{R}_M,0}$. Otherwise, let us assume that $\tau_{\mathcal{R}_P} > 1$. We must distinguish three further cases: (i) $\ell_{\mathcal{R}_P} = \ell_{\mathcal{R}_M} \wedge \tau_{\mathcal{R}_P} = \tau_{\mathcal{R}_M} + 1$: in this case, $X_{\mathcal{R}_M,0}$ contains exactly the clocks that will enter $X_{\mathcal{R}_P,1}$; it is possible to transition backwards from $\mathcal{R}_P$ to $\mathcal{R}_M$ by removing all clocks from $X_{\mathcal{R}_P,1}$ and insert them into $X_{\mathcal{R}_M,0}$; (ii) $\ell_{\mathcal{R}_P} = \ell_{\mathcal{R}_M} + 1 \wedge \tau_{\mathcal{R}_P} = \tau_{\mathcal{R}_M} + 1$: in this case, $X_{\mathcal{R}_M,0}$ contains the clocks in $X_{\mathcal{R}_P,-\ell_{\mathcal{R}_P}} \cup X_{\mathcal{R}_P,1}$; it is possible to transition backwards from $\mathcal{R}_P$ to $\mathcal{R}_M$ by removing all clocks from $X_{\mathcal{R}_P,-\ell_{\mathcal{R}_P}}$ and $X_{\mathcal{R}_P,1}$ and insert them into $X_{\mathcal{R}_M,0}$; (iii) $\ell_{\mathcal{R}_P} = \ell_{\mathcal{R}_M} + 1 \wedge \tau_{\mathcal{R}_P} = \tau_{\mathcal{R}_M}$: in this case, $X_{\mathcal{R}_M,0}$ contains exactly the clocks that will become unbounded in $X_{\mathcal{R}_P,-\ell_{\mathcal{R}_P}}$; it is possible to transition backwards from $\mathcal{R}_P$ to $\mathcal{R}_M$ by removing all clocks from $X_{\mathcal{R}_P,-\ell_{\mathcal{R}_P}}$ and insert them into $X_{\mathcal{R}_M,0}$. Hence, a region of class P can either have one immediate delay predecessor of class M to be added to the (up to two) immediate delay predecessors of class Z (when $\tau_{\mathcal{R}_P} = 1$), or at most three immediate delay predecessor regions of class M when $\tau_{\mathcal{R}_P} > 1$.

Combining the above results (1) and (2), it follows that a region of class P has at most three immediate delay predecessors.
- *Class M*: consider a region $\mathcal{R}_M$ of class M . The immediate delay predecessor regions that can be reached backwards from $\mathcal{R}_M$ belong to class P . Hence, let $\mathcal{R}_P$ be a region of class P such that $\tau_{\mathcal{R}_P} > 1$ (otherwise $\mathcal{R}_P$ would reach a region of class Z) and $\tau_{\mathcal{R}_P} = \tau_{\mathcal{R}_M} + 1$. The only way $\mathcal{R}_M$ is reachable from $\mathcal{R}_P$ is by letting the clocks in set $X_{\mathcal{R}_P,\tau_{\mathcal{R}_P}}$ reach the next unit. Then, it is possible to transition backwards from $\mathcal{R}_M$ to $\mathcal{R}_P$ by removing all the clocks from $X_{\mathcal{R}_M,0}$ and inserting them into $X_{\mathcal{R}_P,\tau_{\mathcal{R}_P}}$, while also decreasing by 1 the integer value of those clocks. It follows that a region of class M has exactly one immediate delay predecessor region of class P .
- *Class U*: consider a region $\mathcal{R}_U$ of class U . The immediate delay predecessor regions that can be reached backwards from $\mathcal{R}_U$ belong to class Z . Hence, let $\mathcal{R}_Z$ be a region of class Z . The only way $\mathcal{R}_U$ is reachable from $\mathcal{R}_Z$ is

when all clocks in $X_{\mathcal{R}_Z,0}$ become unbounded simultaneously. It is possible to transition backwards from $\mathcal{R}_U$ to $\mathcal{R}_Z$ by removing all clocks from $X_{\mathcal{R}_U,-\ell_{\mathcal{R}_U}}$ and inserting them into $X_{\mathcal{R}_Z,0}$. Hence, a region of class U has exactly one immediate delay predecessor region of class Z .

It follows that regions in classes Z, M, U have one immediate delay predecessor; regions in class P have at most three immediate delay predecessors. $\square$

D Proof of Theorem 2

Lemma 1. *Let $\mathcal{A}$ be a TA such that $n = |X_{\mathcal{A}}|$ and let $\mathcal{R}$ be a region of $\mathcal{A}$. If the order in which clocks become unbounded must be tracked, then the number of discrete predecessors of $\mathcal{R}$ over a single transition is at most:*

$$\sum_{u=0}^n \left(\sum_{i=0}^{n-u} (c_m + 1)^i (c_m)^{n-u-i} \binom{n}{n-u-i} \sum_{k=0}^{n-u-i} k! \left\{ \begin{matrix} n-u-i \\ k \end{matrix} \right\} \cdot \binom{n}{u} \sum_{w=0}^u w! \left\{ \begin{matrix} u \\ w \end{matrix} \right\} \right)$$

where $\left\{ \begin{matrix} \alpha \\ \beta \end{matrix} \right\} = \frac{1}{\beta!} \sum_{d=0}^{\beta} (-1)^d \binom{\beta}{d} (\beta - d)^{\alpha}$ is a Stirling number of the second kind, that is, the number of ways to partition a set of α elements into β non-empty subsets, for two given natural numbers $\alpha, \beta \in \mathbb{N}$.

Proof. We assume that all clocks of $\mathcal{A}$ share the same maximum constant c_m . The lemma also applies when each clock has its own maximum constant.

The proof proceeds by enumerating all possible ordered partitions of bounded and unbounded clocks. We consider the following transition over which discrete predecessors of a region $\mathcal{R}$ are computed: $(q, a, \gamma, Y, q_{\mathcal{R}}) \in T$, where the guard is of the form $\gamma := x_1 \geq 0 \wedge \dots \wedge x_n \geq 0$, where $X_{\mathcal{A}} = \{x_1, \dots, x_n\}$, and the set of reset clocks is $Y = X_{\mathcal{A}}$, i.e., all clocks are reset, allowing them to take on any possible value up to c_m and potentially become unbounded.

Let u denote the total number of clocks that are unbounded in the predecessor of $\mathcal{R}$ (for conciseness, in the following we simply say bounded or unbounded).

If $u = n$, meaning that all clocks are unbounded, the total number of ordered partitions is given by: $\sum_{w=0}^u w! \left\{ \begin{matrix} u \\ w \end{matrix} \right\}$. This corresponds to the definition of *Fubini numbers* [14,22], which count the number of ordered partitions of a given set.

If $u = 0$, meaning that all clocks are bounded, the total number of ordered partitions is given by: $\sum_{i=0}^n (c_m + 1)^i (c_m)^{n-i} \binom{n}{n-i} \sum_{k=0}^{n-i} k! \left\{ \begin{matrix} n-i \\ k \end{matrix} \right\}$. The index i represents the total number of clocks that have no fractional part. Since these clocks do not contribute to the ordering, the number of ordered partitions only depends on the remaining $n - i$ clocks, hence the Fubini sum goes up to $n - i$. To also account for all possible ways of choosing which $n - i$ clocks have a fractional part greater than zero, the sum must be repeated over all $\binom{n}{n-i}$ combinations. Two additional terms $(c_m + 1)^i$ and $(c_m)^{n-i}$ account for all possible integer values that clocks can assume: clocks with no fractional part can take any value from 0 to c_m (i.e., $c_m + 1$ possibilities), while the remaining clocks, having a fractional part greater than zero, are limited to the range $[0, c_m)$ (i.e., c_m possibilities).

In the general case, it holds that $0 \leq u \leq n$. The final result is obtained by combining the contributions of both bounded and unbounded clocks for each possible value of u . The contribution from the bounded clocks is given by: $\sum_{i=0}^{n-u} (c_m + 1)^i (c_m)^{n-u-i} \binom{n}{n-u-i} \sum_{k=0}^{n-u-i} k! \left\{ \begin{smallmatrix} n-u-i \\ k \end{smallmatrix} \right\}$. The contribution from the unbounded clocks is: $\binom{n}{u} \sum_{w=0}^u w! \left\{ \begin{smallmatrix} u \\ w \end{smallmatrix} \right\}$, where $\binom{n}{u}$ accounts for the choice of which u clocks are unbounded. Multiplying the bounded and unbounded contributions for each fixed u , and summing over all $u \in [0, n]$, yields the maximum number of discrete predecessors of $\mathcal{R}$ over a single transition. $\square$

Theorem 2. *Let $\mathcal{A}$ be a TA such that $n = |X_{\mathcal{A}}|$ and let $\mathcal{R}$ be a region of $\mathcal{A}$. If the order in which clocks become unbounded must be tracked, then the number of discrete predecessors of $\mathcal{R}$ over a single transition is $O\left(\left(\frac{c_m+1}{\ln 2}\right)^n \cdot (n+1)!\right)$.*

Proof. Let S_n be the formula of Lemma 1.

The value $a_u = \sum_{w=0}^u w! \left\{ \begin{smallmatrix} u \\ w \end{smallmatrix} \right\}$ is the u -th Fubini number. Therefore,

$$\sum_{k=0}^{n-u-i} k! \left\{ \begin{smallmatrix} n-u-i \\ k \end{smallmatrix} \right\} = a_{n-u-i}.$$

Moreover, $(c_m + 1)^i (c_m)^{n-u-i} < (c_m + 1)^{n-u}$. Hence, S_n is less than:

$$S'_n = \sum_{u=0}^n (c_m + 1)^{n-u} \binom{n}{u} a_u \sum_{i=0}^{n-u} \binom{n}{n-u-i} a_{n-u-i}.$$

Given that $a_n = \sum_{i=1}^n \binom{n}{i} a_{n-i}$, then $\sum_{i=0}^{n-u} \binom{n}{n-u-i} a_{n-u-i} = 2a_{n-u}$. Therefore, S'_n is equal to:

$$\begin{aligned} 2 \sum_{u=0}^n (c_m + 1)^{n-u} \binom{n}{u} a_{n-u} a_u &\leq 2 \sum_{u=0}^n (c_m + 1)^{n-u} \cdot \sum_{u=0}^n \binom{n}{u} a_{n-u} a_u \\ &\leq 2(c_m + 1)^{n+1} \cdot \sum_{u=0}^n \binom{n}{u} a_{n-u} a_u. \end{aligned}$$

We want to estimate $T_n = \sum_{u=0}^n \binom{n}{u} a_{n-u} a_u$, when $n \rightarrow \infty$. Fubini numbers satisfy the asymptotic estimate $a_i = O\left(\frac{i!}{(\ln 2)^i}\right)$. Using this estimate, we write:

$$\binom{n}{u} a_{n-u} a_u = O\left(\binom{n}{u} \cdot \frac{(n-u)!}{(\ln 2)^{n-u}} \cdot \frac{u!}{(\ln 2)^u}\right) = O\left(\binom{n}{u} \cdot (n-u)! \cdot u! \cdot \frac{1}{(\ln 2)^n}\right).$$

Since $\binom{n}{u} \cdot (n-u)! \cdot u! = n!$, each term in the sum T_n is $O\left(\frac{n!}{(\ln 2)^n}\right)$.

There are $n+1$ such terms, hence T_n is:

$$O\left((n+1) \cdot \frac{n!}{(\ln 2)^n}\right) = O\left(\frac{(n+1)!}{(\ln 2)^n}\right)$$

i.e., S_n is $O\left(\left(\frac{c_m+1}{\ln 2}\right)^n \cdot (n+1)!\right)$. $\square$

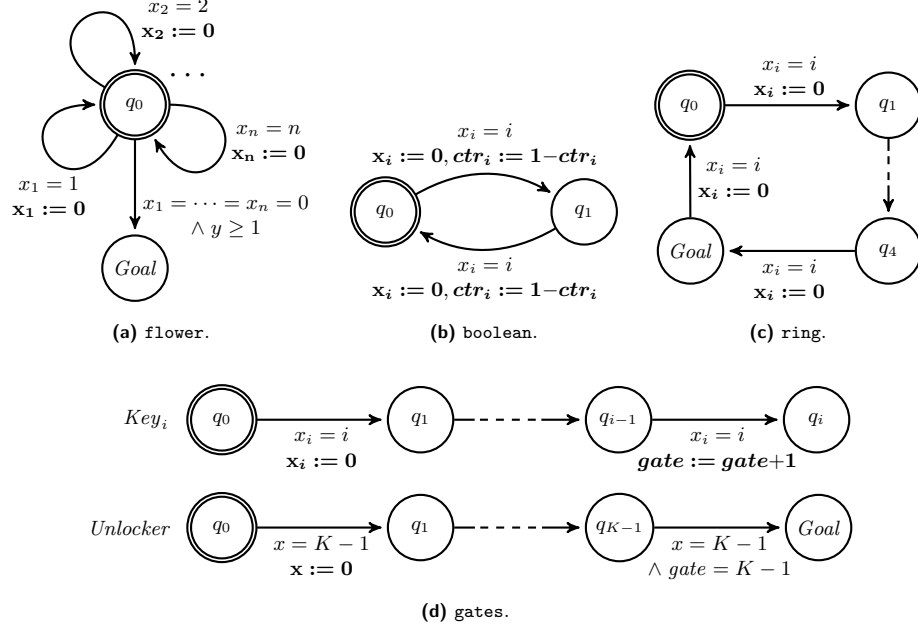


Fig. 3: Depiction of the benchmark TA used in both Table 1 and Table 6. **boolean**, **ring**, and **gates** are manually constructed networks of TA, whereas **flower** is a single TA adapted from [21] (it is included here for completeness).

E Complete Experimental Evaluation

This section provides a detailed description of the manually constructed benchmarks and reports the complete version of the tables discussed in Section 5.

Tables report Verification Time (VT), Execution Time (ET), memory (Mem, representing the Maximum Resident Set Size), and the number of stored regions and states. ET measures the time elapsed between issuing a verification command and receiving the result from a tool. Timings are in seconds, memory in MB. The size of a network is denoted by K (for **flower**, K indicates the number of clocks). All values are averages over five runs, with a timeout of 600 s set on ET. Timeouts are reported as TO, ε denotes times below 0.001 s, and out-of-memory events are denoted by OOM. Bold cells indicate the fastest VT (on ties, the fastest ET). KO indicates that an error occurred during verification.

E.1 Punctual guards benchmarks

Figure 3 depicts the manually created benchmarks used in Table 1 and Table 6. Those are manually constructed networks of TA, except for **flower**, which is a single TA adapted from [21].

Flower The **flower** TA (Figure 3a) has two locations: q_0 (initial) and $Goal$, and it includes $n + 1$ clocks, denoted by $x_1, \dots, x_n, y$. Over q_0 there are n self-

loops, one for each clock x_i . In each self-loop, x_i is compared to the constant i (i.e., the guard is $x_i = i$) and reset along such transition. To reach *Goal*, all clocks x_i must simultaneously be exactly zero (with no fractional part), while y must satisfy $y \geq 1$. Consequently, the integer value of y when transitioning to *Goal* is equal to the least common multiple of $1, 2, \dots, n$.

Boolean The **boolean** network consists of K TA of the form shown in Figure 3b. For every index i , with $1 \leq i \leq K$, the i -th TA flips the value of the integer variable ctr_i between 0 and 1 whenever it transitions between location q_0 and location q_1 . All integer variables are initialized to 0 at the start of execution. The query $\exists \Diamond(ctr_1 = 1 \wedge ctr_2 = 1 \wedge \dots \wedge ctr_K = 1)$ is satisfied only if every variable ctr_i is set to 1. Since TChecker does not support integer variables in reachability queries, an equivalent property was formulated using locations: $\exists \Diamond(Boolean_1.q_1 \wedge Boolean_2.q_1 \wedge \dots \wedge Boolean_K.q_1)$.

Ring The **ring** network consists of K TA of the form shown in Figure 3c. For brevity, the dashed edge abbreviates the additional locations and identical transitions that are not depicted (each TA includes 6 locations and 6 transitions).

Gates The **gates** network consists of $K - 1$ Key_i TA, for $1 \leq i \leq K - 1$, and an *Unlocker* TA, as depicted in Figure 3d. For brevity, the dashed edges abbreviate additional locations and identical transitions that are not depicted. The total number of locations and transitions depends on the index i for each Key_i and on K for the *Unlocker*. Each Key_i increments the integer variable *gate* by 1 during its last transition. The *Unlocker* can reach the *Goal* location only if *gate* equals $K - 1$, allowing its last transition to fire.

E.2 Full benchmark tables

Table 6, Table 7, Table 8, and Table 9 present the extended version of the benchmarks discussed in Section 5. In particular, Table 9 includes an additional benchmark on the Lynch-Shavit protocol, similar to the Fischer case.

Symmetry reduction Table 10 shows benchmarks in which the symmetry reduction optimization was enabled in TARZAN and Uppaal (symmetry reduction cannot be manually controlled in TChecker); all symmetric TA were identical. Each *Viking_i* had 20 as its maximum constant. The **trainAHV93** benchmark was executed using BFS, as this exploration strategy yielded the best performance across all three tools. Even though TARZAN is not the fastest in VT, symmetry reduction makes these benchmarks tractable with regions. Indeed, without this optimization, TARZAN ran out of memory in **trainAHV93** for $K = 10, 14, 18, 22$, and in **bridge** for $K = 17, 21$. Interestingly, the results suggest that TChecker does not apply specific optimizations in **trainAHV93**, but it does in **bridge**.

Additional benchmarks Table 11 reports additional benchmarks that were conducted. These benchmarks include both small and medium-sized networks of TA, as well as single TA. The **exSITH** and **simple** benchmarks consist of a single TA. For **simple**, the maximum constant was varied (10, 100, and 1000). Apart from the **soldiers** benchmark, TARZAN performs comparably to both Uppaal and TChecker in terms of VT, ET, and memory usage. The query $\exists \Diamond(false)$ required to compute the entire reachable state space.

Table 6: When all transitions have punctual guards, regions may outperform zones. Here, **flower** is a single TA, while the others are networks of TA.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► boolean [26] :: [Query: $\exists \Diamond(ctr_1 = 1 \wedge ctr_2 = 1 \wedge \dots \wedge ctr_K = 1)$] ◀												
2	ϵ	0.013	7.62	9	ϵ	0.005	5.92	6	ϵ	0.051	18.80	6
4	ϵ	0.013	7.78	24	ϵ	0.007	6.47	185	0.001	0.052	18.85	185
6	0.003	0.016	13.42	425	0.088	0.096	13.80	8502	0.075	0.128	19.91	8502
8	0.013	0.027	29.10	1009	75.461	75.980	220.82	566763	77.204	77.523	81.79	566763
10	0.070	0.086	110.29	4050	—	TO	—	—	—	TO	—	—
12	0.341	0.361	476.73	15961	—	TO	—	—	—	TO	—	—
14	0.737	0.764	965.29	28679	—	TO	—	—	—	TO	—	—
16	1.578	1.616	1788.46	43919	—	TO	—	—	—	TO	—	—
► flower [21] :: [Query: $\exists \Diamond(Flower.Goal)$] ◀												
3	ϵ	0.013	7.64	13	ϵ	0.005	5.84	7	ϵ	0.055	18.80	7
5	ϵ	0.013	7.92	95	0.001	0.007	6.35	150	0.001	0.056	18.82	150
7	ϵ	0.022	10.65	573	0.322	0.330	13.12	3508	0.399	0.455	19.76	3508
9	0.007	0.022	58.62	9161	—	TO	—	—	—	TO	—	—
11	0.028	0.044	177.34	30761	—	TO	—	—	—	TO	—	—
13	0.412	0.450	2034.18	370331	—	TO	—	—	—	TO	—	—
15	7.188	7.398	12900.36	5142671	—	TO	—	—	—	TO	—	—
17	16.212	16.562	12942.48	11023829	—	TO	—	—	—	TO	—	—
► gates [26] :: [Query: $\exists \Diamond(Unlocker.Goal)$] ◀												
3	ϵ	0.013	7.79	26	ϵ	0.006	5.99	18	ϵ	0.054	18.86	18
5	ϵ	0.013	8.84	76	0.001	0.008	7.19	549	0.001	0.055	19.01	549
7	ϵ	0.014	11.23	151	0.110	0.122	21.08	32979	0.035	0.092	22.79	32979
9	0.001	0.015	15.43	251	47.895	48.771	806.72	3071646	5.228	6.509	451.71	3071646
11	0.002	0.016	21.87	379	—	TO	—	—	—	TO	—	—
13	0.004	0.018	31.31	537	—	TO	—	—	—	TO	—	—
15	0.006	0.020	44.01	708	—	TO	—	—	—	TO	—	—
17	0.009	0.023	60.85	922	—	TO	—	—	—	TO	—	—
► ring [26] :: [Query: $\exists \Diamond(P_0.Goal \wedge P_1.Goal \wedge \dots \wedge P_K.Goal)$] ◀												
2	ϵ	0.013	7.84	59	ϵ	0.006	6.14	45	ϵ	0.054	18.83	45
4	0.001	0.015	12.40	542	0.020	0.029	11.56	7738	0.010	0.065	19.52	7738
6	0.011	0.025	48.96	3397	12.137	12.561	268.93	1180575	3.406	3.715	128.87	1180575
8	0.204	0.225	687.77	47063	—	TO	—	—	—	TO	—	—
10	0.272	0.295	897.00	52669	—	TO	—	—	—	TO	—	—
12	6.901	7.097	13049.36	1067623	—	TO	—	—	—	TO	—	—
14	—	—	OOM	—	—	TO	—	—	—	TO	—	—
16	—	—	OOM	—	—	TO	—	—	—	TO	—	—

Table 7: The results are derived from networks of closed TA, where guards may either be punctual or contain non-strict inequalities. In this setting, TARZAN can handle both large networks (`medicalWorkflow`) and large constants (`mpeg2`).

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► <code>medicalWorkflow</code> [32] :: [Query: $\exists\Diamond(Patient_1.Done \wedge Patient_2.Done \wedge \dots \wedge Patient_{K-2}.Done)$] ◀												
12	0.007	0.021	33.39	660	0.011	0.030	22.63	639	0.002	0.059	20.83	582
22	0.056	0.072	204.97	2715	0.316	0.560	173.75	3774	0.032	0.093	31.76	2862
32	0.218	0.241	802.55	7170	2.693	3.591	947.61	11409	0.232	0.305	79.99	7842
42	0.609	0.647	2271.65	15025	13.571	16.096	2517.18	25544	1.275	1.379	230.07	16522
52	1.432	1.495	5210.36	27280	48.690	55.370	5724.98	48179	5.524	5.695	598.03	29902
62	2.860	2.983	10440.43	44935	141.818	156.903	10778.80	81314	19.325	19.648	1366.31	48982
72	5.781	5.959	12891.17	68990	355.827	389.223	16109.45	126949	54.200	54.838	2799.57	74762
82	10.622	10.889	13519.50	100445	—	TO	—	—	133.114	134.367	5257.41	108242
92	17.877	18.264	13811.47	140300	KO	KO	KO	KO	293.285	295.567	9219.25	150422
102	27.918	28.466	13780.57	189555	KO	KO	KO	KO	593.317	597.306	12364.67	202302
► <code>mpeg2</code> [32] :: [Query: $\exists\Diamond(BFrame_1.Bout \wedge BFrame_2.Bout \wedge \dots \wedge BFrame_{K-4}.Bout)$] ◀												
8	0.365	0.385	451.46	26916	0.003	0.014	8.96	531	0.001	0.058	19.46	537
12	0.796	0.819	718.63	27238	8.405	8.663	252.70	383103	1.621	1.766	112.33	383109
16	1.364	1.392	1032.46	27704	—	TO	—	—	—	TO	—	—
20	2.521	2.558	1575.82	28314	—	TO	—	—	—	TO	—	—
24	3.865	3.910	2085.50	29068	—	TO	—	—	—	TO	—	—

Table 8: Large constants negatively impact the VT of regions, unlike zones. Here, the first row of `pagerank` corresponds to the variant with truncated constants.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► <code>CSMA/CD</code> [23] :: [Query: $\exists\Diamond(P_1.senderRetry \wedge P_2.senderRetry \wedge P_3.senderTransm) \wedge P_4.senderRetry \wedge \dots \wedge P_r.senderRetry$] ◀												
21	0.198	0.215	364.96	20936	0.004	0.038	30.32	232	0.001	0.061	20.24	269
23	0.262	0.281	482.81	25422	0.005	0.053	40.75	254	0.001	0.061	20.10	318
26	0.388	0.409	700.02	32931	0.008	0.079	62.17	287	0.002	0.062	20.64	399
31	0.674	0.701	1194.45	47526	0.018	0.218	117.03	371	0.002	0.064	21.11	554
51	3.364	3.431	5351.35	131906	0.293	1.386	837.12	1381	0.010	0.077	23.89	1424
► <code>pagerank</code> [5] :: [Query: $\exists\Diamond(Stage_7.Completed)$] ◀												
9	0.068	0.085	179.33	4193	0.002	0.086	58.54	1129	0.002	0.076	22.89	1129
9	4.227	4.361	10950.36	257311	0.002	0.085	58.53	1129	0.002	0.076	23.01	1129

Table 9: Safety properties require TARZAN to visit the entire reachable state space.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► <code>fischer</code> [34] :: [Query: $\exists\Diamond(Fischer_1.cs \wedge Fischer_2.cs)$] ◀												
2	0.001	0.015	9.60	542	ϵ	0.006	6.01	18	0.001	0.055	18.87	18
3	0.025	0.039	55.62	9324	ϵ	0.006	6.26	65	0.001	0.055	18.91	65
4	0.644	0.669	1098.36	180032	ϵ	0.007	6.75	220	0.001	0.055	18.96	220
5	19.627	19.849	12948.99	3920652	0.004	0.011	9.30	727	0.002	0.057	19.07	727
6	—	—	OOM	—	0.024	0.032	12.25	2378	0.008	0.064	19.43	2378
► <code>lynch</code> [17] :: [Query: $\exists\Diamond(P_1.CS7 \wedge P_2.CS7)$] ◀												
2	0.076	0.091	152.12	32268	ϵ	0.007	6.19	38	ϵ	0.058	18.95	38
3	20.137	20.373	12857.20	5289449	ϵ	0.008	6.49	125	0.001	0.056	19.05	125
4	—	—	OOM	—	0.001	0.010	7.38	380	0.001	0.057	19.15	380

Table 10: Symmetry reduction renders some benchmarks tractable with regions. For each benchmark, symmetric TA are identical. The maximum constant for each $Viking_i$ in **bridge** is 20. The **trainAHV93** benchmark was run in BFS.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► trainAHV93 [17] :: [Query: $\exists\Diamond(\text{Controller.controller3})$] ◀												
6	0.028	0.042	57.87	3803	0.003	0.011	7.89	970	ε	0.055	19.09	29
10	0.227	0.249	261.72	18672	1.993	2.021	91.89	110096	0.001	0.056	19.17	30
14	0.553	0.572	370.96	33907	159.005	159.662	2347.05	2087686	0.001	0.059	19.26	30
18	0.977	0.997	464.34	49143	—	TO	—	—	0.001	0.057	19.30	30
22	1.520	1.542	559.03	64379	—	TO	—	—	0.002	0.057	19.40	30
► bridge [34] :: [Query: $\exists\Diamond(Viking_1.safe \wedge Viking_2.safe \wedge \dots \wedge Viking_{K-1}.safe)$] ◀												
5	0.004	0.017	19.00	852	ε	0.006	6.24	41	ε	0.055	19.03	23
9	0.018	0.033	68.88	2271	ε	0.008	6.44	149	0.001	0.057	19.07	55
13	0.045	0.060	158.88	3883	ε	0.011	6.97	321	0.001	0.056	19.13	87
17	0.086	0.104	286.59	5687	0.002	0.017	17.07	557	0.002	0.058	19.17	119
21	0.144	0.163	448.89	7683	0.004	0.030	31.56	857	0.003	0.060	19.26	151

Table 11: In small to medium-sized benchmarks, regions perform comparably with zones. Here, **exSITH** and **simple** are single TA.

K	TARZAN				TChecker				UPPAAL			
	VT	ET	Mem	Regions	VT	ET	Mem	States	VT	ET	Mem	States
► andOrOriginal [17] :: [Query: $\exists\Diamond(false)$] ◀												
3	0.001	0.016	14.50	640	ε	0.007	6.35	9	ε	0.052	18.96	8
► exSITH [17] :: [Query: $\exists\Diamond(A.qBad)$] ◀												
1	ε	0.013	8.10	225	ε	0.006	5.87	4	ε	0.054	18.79	4
► latch [17] :: [Query: $\exists\Diamond(Latch_1.LatchDIE1)$] ◀												
7	0.009	0.024	59.34	2025	ε	0.008	6.36	7	ε	0.056	19.03	7
► maler [17] :: [Query: $\exists\Diamond(Job_1.End1 \wedge Job_2.End2 \wedge Job_3.End3 \wedge Job_4.End4)$] ◀												
4	ε	0.014	8.38	114	ε	0.008	6.35	75	ε	0.056	19.11	75
► rcp [17] :: [Query: $\exists\Diamond(S1o.S1oEnd)$] ◀												
5	0.001	0.014	12.25	346	ε	0.010	6.50	27	0.001	0.056	19.18	18
► simple [20] :: [Query: $\exists\Diamond(false)$] ◀												
1	ε	0.013	7.64	56	ε	0.006	5.81	3	ε	0.053	18.76	3
1	ε	0.013	7.92	428	ε	0.006	5.90	3	ε	0.054	18.76	3
1	0.001	0.015	12.28	4028	0.001	0.007	6.52	3	0.001	0.054	18.83	3
► soldiers [17] :: [Query: $\exists\Diamond(E.Escape)$] ◀												
5	0.128	0.146	494.43	24194	ε	0.007	6.45	93	ε	0.056	19.05	93
► srLatch [17] :: [Query: $\exists\Diamond(Env.envFinal)$] ◀												
3	ε	0.013	7.62	9	ε	0.007	6.24	5	ε	0.052	19.05	4